Received: 30 April 2019 | Revised: 21 May 2019 | Accepted: 30 May 2019
DOI: 10.1002/ett.3675

SPECIAL ISSUE ARTICLE

WILEY

Android malware detection through generative adversarial networks

Muhammad Amin[1,3] | **Babar Shah**[2] | **Aizaz Sharif**[3] | **Tamleek Ali**[1] | **Ki-lL Kim**[4] | **Sajid Anwar**[1]

[1]Department of Computer Science, Institute of Management Sciences, Peshawar, Pakistan

[2]College of Information Technology, Zayed University, Dubai, United Arab Emirates

[3]Department of Computer Science, National University of Computer and Emerging Sciences, Peshawar, Pakistan

[4]Department of Computer Science and Engineering, Chungnam National University, Daejeon, South Korea

Correspondence
Muhammad Amin, Department of Computer Science, Institute of Management Sciences, Peshawar-25100, Pakistan; or Department of Computer Science, National University of Computer and Emerging Sciences, Peshawar-25100, Pakistan.
Email: muhammad.amin@nu.edu.pk

Present Address
Muhammad Amin, 160 Industrial Estate, Jamrud Road, Peshawar-25100, Pakistan.

Funding information
Zayed University-UAE, Grant/Award Number: RIF:R18051

Abstract

Mobile and cell devices have empowered end users to tweak their cell phones more than ever and introduce applications just as we used to with personal computers. Android likewise portrays an uprise in mobile devices and personal digital assistants. It is an open-source versatile platform fueling incalculable hardware units, tablets, televisions, auto amusement frameworks, digital boxes, and so forth. In a generally shorter life cycle, Android also has additionally experienced a mammoth development in application malware. In this context, a toweringly large measure of strategies has been proposed in theory for the examination and detection of these harmful applications for the Android platform. These strategies attempt to both statically reverse engineer the application and elicit meaningful information as features manually or dynamically endeavor to quantify the runtime behavior of the application to identify malevolence. The overgrowing nature of Android malware has enormously debilitated the support of protective measures, which leaves the platforms such as Android feeble for novel and mysterious malware. Machine learning is being utilized for malware diagnosis in mobile phones as a common practice and in Android distinctively. It is important to specify here that these systems, however, utilize and adapt the learning-based techniques, yet the overhead of hand-created features limits ease of use of such methods in reality by an end user. As a solution to this issue, we mean to make utilization of deep learning–based algorithms as the fundamental arrangement for malware examination on Android. Deep learning turns up as another way of research that has bid the scientific community in the fields of vision, speech, and natural language processing. Of late, models set up on deep convolution networks outmatched techniques utilizing handmade descriptive features at various undertakings. Likewise, our proposed technique to cater malware detection is by design a deep learning model making use of generative adversarial networks, which is responsible to detect the Android malware via famous two-player game theory for a rock-paper-scissor problem. We have used three state-of-the-art datasets and augmented a large-scale dataset of opcodes extracted from the Android Package Kit bytecode and used in our experiments. Our technique achieves F1 score of 99% with a receiver operating characteristic of 99% on the bytecode dataset. This proves the usefulness of our technique and that it can generally be adopted in real life.

© 2019 John Wiley & Sons, Ltd.

1 | INTRODUCTION

Mobile phones have not only diminished the window between the capacities of telephones and that of personal computers (PCs) but also outperformed them in a few zones by uniting several imaginative avenues, such as worldwide positioning (GPS), navigation and maps, top notch cameras, and a variety of sensors that are not generally incorporated into PCs. Truth be told, a large portion of the conventional specialized computing applications are currently progressively relocating to cell phones, for example, email and person-to-person communication via social networks. Android is a Google's virtual-machine–based portable operating system. It is an open-source software stack based on Linux kernel. Initially designed for smartphones and tablets, further development in Android resulted in its widespread use for televisions, cars (Android Auto), and wristwatches (Android Wear). With its first commercial release in September 2008, it has progressed in leaps and bounds, which eventually lead to its latest release of version 8.1 Oreo in December 2017. Google's CEO announced that Android now has more than two billion monthly active devices and has around one billion users. Google Play Store, which is Google's collection of Android applications (apps), has 3.5 million apps, and in 2016 alone, people downloaded around 82 billion apps for Android. The ubiquity and acceptance of mobile phones have extraordinarily fortified the spread of malware, particularly on the prominent hosts, for example, Android. A notable element of the Android platform is that applications are accessible not only from the local Google Play Store but also through an assortment of third-party application sources. While applications downloaded and consumed from the local application store can justifiably be trusted, the same cannot be said in regards to the applications accessed from arbitrary third-party sources.[1]

Malware is a short term for an application exhibiting malicious behavior. Essentially, malware is any bit of programming that is created to do damage to information, gadgets, or end user. These malware apps often download malicious programs to the user's system without the knowledge and acknowledgment of the user. They typically infect the host machine by tricking the user into clicking and/or installing a program that they should not from the Internet. After the installation, the malicious code or script executes, and actions, which the user did not intend or anticipated, occur. On the other hand, applications, which are not harmful and perform intended actions with the knowledge and acknowledgment of the end user on a host system, are known as benign applications. Note that malware is a broad term for the various different applications, which exhibit malicious traits. According to Kaspersky Lab, 5 730 916 malicious installation packages, 94 368 mobile banking Trojans, and 544 107 mobile ransomware Trojans were detected in 2017 for smartphones.[2] However, 97% of the total malware for mobile phones is found in Android's ecology.[3]

The adaptability of Android through custom firmware and arbitrary application sources joined by the sensitive nature of the user data stored on these gadgets requires improvement and execution of bleeding edge safety efforts to guarantee the accessibility, privacy, and rectitude of user information, administration, applications, systems, and importantly the host itself so as to capture the malware. The expanding and enhanced nature of Android malware has vastly lessened the efficacy of active defensive measure, which leaves platforms such as Android defenseless for novel and anonymous malware.[4]

Anti-malware applications[5,6] have a simple job of analyzing and distinguishing between malware and benign apps. Earlier, analysis of malware for a smartphone like Android was guileless,[7-9] since hostile apps exhibited uncomplicated malign traits, which could just be identified by watching the activities they performed. Of late, malware for Android has dynamically turned out to be unusual and have selected the more forceful procedures.[10] As a response, specialists in malware security have proposed an array of identification techniques utilizing machine learning–based algorithms. However, our ability is to a great extent limited by the constrained comprehension of these never ceasing application malware such as zero-day vulnerability.[9] Alarmingly, malware creators have managed to upload malignant applications to the official Android Market,[2,11] sitting tight for naive clients to download and use them. Conventional machine learning–based anti-malware techniques[12] suffer from twofold of the problems. First, the input space is to be analyzed manually, which is very laborious, assumes a lot of expertise to work with, and eventually might lead to draw vague conclusions. Second, the features are to be created by hand using predefined patterns, where there are chances that some useful information is left unfocused, which could play a prominent role in vetting the application's behavior either as malicious or benign.

Handcrafted features[13] are the properties that we derive from the input by using different algorithms. Handcrafted features are used with machine learning approaches for recognizing an object with classifiers such as support vector machines (SVMs).[14] New approaches such as deep neural networks do not have to be supplied with handcrafted features for analysis and classification of application behavior.[15] Rather, for example, starting from loading an unlabeled dataset to the network, finding appropriate patterns, creating and extracting useful information-like features, and finally classifying the application as belonging to either class are all automatic; thus, no human intervention is required in any step. In

fact, all the human effort from start to end is offloaded to the machines to carry out all the steps. This notion of neural networks[16] being automatic from start to end is usually referred to as "end-to-end" architecture,[13] where any or all the problems faced by conventional machine learning–based approaches can reasonably be avoided. At a number of recent occasions, deep learning–based approaches have outperformed the traditional learning–based mechanisms specifically in the fields of vision and natural language processing.[17,18] In addition, when input space tends to grow very large, it seems computationally infeasible for traditional learning algorithms to scale with respect to the size of input data, while deep learning[19] algorithms naturally assume a very large input dataset to learn and predict the output form.

In this paper, we therefore opt to use the deep learning–based algorithm for the analysis and prediction of malware in the domain of Android. We have used generative adversarial networks (GANs), which were proposed in the work of Goodfellow et al.[20] In GANs, two separate neural nets are run in a competition parallel to each other for resolving conflict and cooperation scenario. In our case, one network tries to generate the malware while the other network tries to detect the malware generated as being real or fake. Both the neural nets try to win this game of generation and detection. This particular scenario resembles the rock-paper-scissors problem,[21] which is resolved through the two-player game theory using the optimization known as the Nash equilibrium.[22] This also resembles the police-and-thief game scenario where each player tries to improve its tactics against each other up to a point where none of the players can improve further without changing the opponent's strategy.

Ever since they have been proposed, GANs have seen a large number of adaptations to various domains due to their ability to scale with large input sizes, being computationally fast, and being precise in their predictions. For a quick and brief review, Section 5 can be consulted. Because of the same reasons, we have applied GANs to the malware detection for Android.

Layout of the paper. The rest of the paper is structured as followed: Section 2 describes the necessary background. Section 3 discusses the literature review regarding GAN architecture.

Section 4 extensively discusses the related approaches and provides a brief comparison to our work. Section 5 is based on GAN architecture. Section 6 provides information about the datasets we are working on. Section 7 discusses the proposed architecture. Section 8 describes the experimental setup, and Sections 8.1 provide the details of the environment and tools. Generators and adversarial networks are mentioned in Sections 9.1 and 9.2, respectively. Section 9 gives insight about training GANs. Section 10 is all about the results and comparison, and Section 11 gives the conclusion and future work.

2 | BACKGROUND

Here, we provide the requisite information as background, which will help the readers better understand this research work.

2.1 | Neural network

A neural network is an approximation model inspired by the human brain. Real-world data is given as input to these neural networks so that features of the data can be learned and the label can be predicted as a result. There are many steps involved in the neural network as discussed below, and the overall architecture can be shown in Figure 1.

2.2 | Input layer

The neural network's starting layer is called an input layer, which contains the data from an image, a text file, or any other source. The inputs are obtained from a dataset, which can be either in a processed or unprocessed form. If the data is preprocessed, then it will be in the form of rows and columns, where each row is a data point containing column values as features of that data point. These features are then fed one by one to the input layer for neural network training. If the dataset is not in a machine learning–ready state, then it first goes through the data preparation stage to obtain features of each input. The size of features (column size) is actually the size of the input layer neurons, which are multiplied with weights to pass on to the next layer called the hidden layer.

2.3 | Hidden layer

The hidden layer is where all the magic happens. At the start, the neural network was supposed to work on one or at most two hidden layers between the input and output layers. However, the revolution came in the form of deep neural

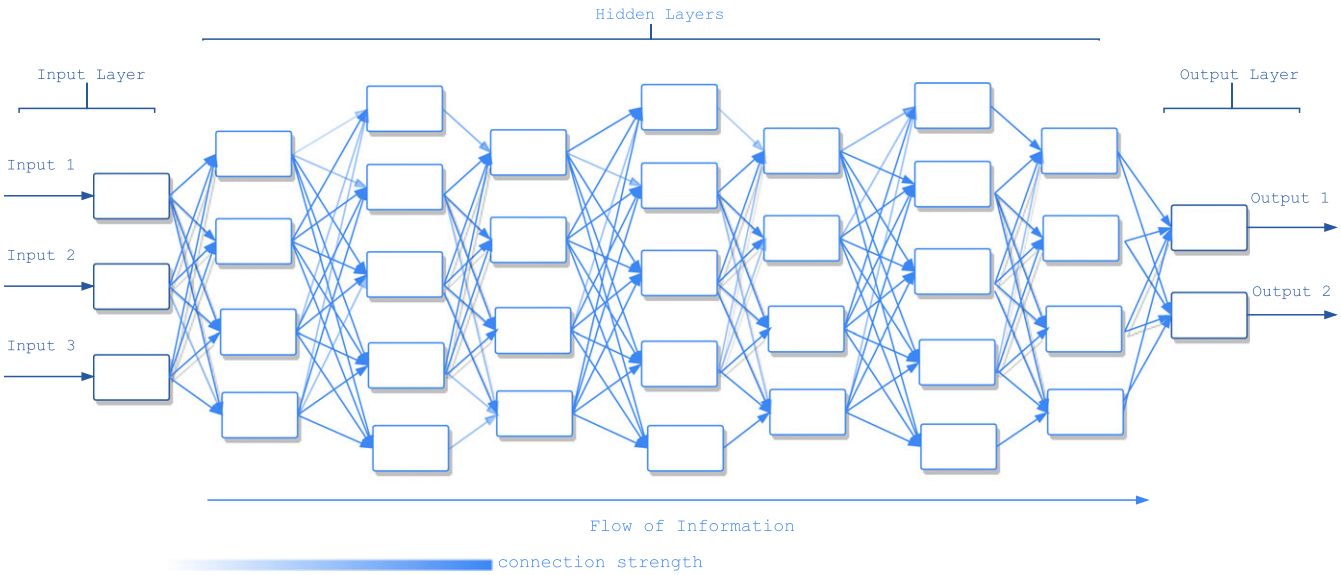

FIGURE 1 A fully connected neural network architecture

networks where the number of hidden layers has been increased, and it resulted in achieving huge improvements in various applications and domains of artificial intelligence. The hidden layer contains neurons having activation functions in them. They perform two tasks: (1) Receive the input from the input layer or previously hidden layer in the form of "input × weights + bias" and apply activation functions to bring them in a particular range, and then (2) act as an input layer for the next hidden layer in case of a dense neural network. The mathematical form of a particular hidden layer can be expressed as follows:

$$h(x) = \Theta_0 + \Theta_1 x,$$

where h refers to the given hidden state, x refers to the input of a given hidden layer, and θ_0 and θ_i are the weights being calculated.

2.4 | Activations

Activation functions are used inside each neuron of a hidden layer and an output layer to decide whether the coming input from the previous layer is enough to activate this neuron, represented by one, or it will remain off, represented by zero. The neuron is activated if it is above some decided threshold; else, it will not. It is added in the neural networks to the algorithm to illustrate the presence of some distinguished features of a given input.

There are many activation functions used in machines, especially in deep learning. The most basic one is a step function, where the input after multiplying with weights and adding bias is passed through the step function and it acts as a binary value. The step function either makes the neuron 1 or 0 by passing through some threshold. This had some flaws in terms of multiclass dataset where more than one neuron having 1 will make it harder for the decision-making and learning process. To counter this issue, many activation function types have been proposed, which are discussed below.

2.4.1 | Linear function

It involves a linear function where the activation is equal to the summed input from the previous input multiplied with weights. It cannot be generalized due to the flaw in linear functions where it adds linearity to the neural network, and we want nonlinear behaviors for complex problem solving. The linear function can be expressed as

$$f(x) = wx + b.$$

2.4.2 | Sigmoid function

Sigmoid functions[23] are the most commonly known and widely used activation function in hidden layers. The output of this activation is always either (0, 1) no matter how large the input is. The sigmoid functions suffer from the problem that

they produce very small value Y with respect to X, which results in the "vanishing gradient" problem.[24,25] As a result, the gradients will not be learned, and the neural network will not progress any further. It has still been widely used but with some fine-tuning. The standard sigmoid function is expressed as

$$f(x) = 1/1 + e^{-x},$$

where x is the input to the function.

2.4.3 | Tanh function

Tanh activation function[26] is also similar to the sigmoid function. They produce the output in the range of (-1, 1). Tanh functions also face a vanishing gradient issue at some stage of neural training. The standard tanh function is expressed as

$$f(x) = 1 - \exp^{-2x}/1 + \exp^{-2X},$$

where x is the input to the equation here.

2.4.4 | ReLU function

A rectified linear unit (ReLU)[27] activation function gives output X if the input value is positive; else, it will give zero. It is also a nonlinear function commonly used these days in deep learning algorithms. The main advantage of ReLU is to handle the sparsity in the data. Instead of firing all neurons as active for a decision, it makes half of the neurons zero (since the negative input is marked as zero). This way, only fewer neurons will be participating in decision-making, thus making the loss go down quickly. The only problem is that since some of the neurons are zero when receiving a negative input, during backpropagation, the gradient will not be calculated for improving weights at all. The problem is known as the dying ReLU problem.[28] The standard ReLU function is expressed as

$$R(x) = \max(0, x).$$

2.4.5 | Softmax function

Softmax[29] functions are used on output layer neurons for logistic classification problems. It brings the values from the last hidden layer as input to the form where all the values will be summed to one. The maximum value neuron will help the network choosing the desired class as its final decision. The standard softmax function is expressed as

$$h\theta(x) = 1/1 + \exp(-\theta^T x).$$

2.5 | Regularization and hyperparameters

In machine leaning, the main goal is generalizability. For achieving that usually, both the dimensions of input data are increased to raise the complexity and to make it nonlinear. Nevertheless, doing that puts the data into the problem of being biased, where the network starts learning too much for certain features. Consequently, this issue can be handled using regularization values, which punishes the neural network for learning parameters too large, making the model away from being overfit. There are also some variables called hyperparameters[30] that we have to set before starting the neural network training on a dataset. These parameters have different values at the experimental basis, mainly decided after the type of dataset that we are dealing with. There are two types of hyperparameters: (1) optimizer hyperparameters and (2) model hyperparameters. Optimizer hyperparameters are related to variables optimizing and training a process rather than the model. These involve batch size, the number of epochs, learning rate, etc. Model hyperparameters are more involved in the structuring of the model. They can be like the number of hidden layers, the shape of neural networks, etc.

2.6 | Optimization

Optimization is the process of minimizing the error function calculated in the output layer by modifying the weights of hidden layers during backpropagation. There are many ways for optimization, but the most common technique is calculating gradient descent[31] using derivatives at each layer to obtain the slopes and to help weights learn the parameters for better performance in the next iteration. Optimization is the key to training a process in machine learning to reduce the cost to a local minimum value.

2.7 | Dropouts

Dropout is a technique common in neural networks and used in every deep learning model these days due to its promising effects. It is the process of dropping out (or turning off) some neurons with the probability of 1-p and turning a few on with a probability of p. This process carried out in each epoch, and the main purpose of dropouts is to prevent overfitting.

2.8 | Output layer

The last layer of the neural network and any deep learning model is the output layer, which contains the number of neurons to the classes of our dataset (in terms of supervised learning). The values of the last hidden layer are brought to the output layer using softmax functions, and then in case of logistic regression, the values are turned into one-hot encoding to see which neurons are active for the class representation, as shown in Figure 1. The error between the actual model's output is also calculated here, and backpropagation begins here.

2.9 | Long short-term memory

Common sequences are modeled using recurrent neural networks (RNNs).[32,33] They have two sources of input, ie, the present and the recent past, which are combined to determine how they respond to unseen data, much as we do in real life. Although an RNN is able to keep track of past activations,[34] there are certain issues related to RNNs, such as vanishing (and exploding) gradients.[25] This deters their practicality in the real world. As a remedy to this problem, long short-term memory (LSTMs)[35] are used. The model behind the GAN we are using is LSTM, which is the modified form of an RNN. LSTMs contain information outside the normal flow of the recurrent network in a cell called gate. LSTM has blocks capable of passing information through them, intuitive to real-life gates; hence, they are termed as gates. LSTM controls input flow to cells through the gates. A gate is a collection of mathematical computations akin to neural networks with sigmoid activations and multiplications carried out in a pointwise manner. These can be summarized as follows:

$$h^t = \sigma(Wx^t + Uh_{t-1}), \tag{1}$$

$$\text{remember}^t = \sigma(W_r x^t + U_r wm_{t-1}), \tag{2}$$

$$\text{ltm}^t = \sigma(W_l x^t + U_l wm_{t-1}), \tag{3}$$

$$\text{save}^t = \sigma(W_s x^t + U_s wm_{t-1}), \tag{4}$$

$$\text{focus}^t = \sigma(W_f x^t + U_f wm_{t-1}). \tag{5}$$

Here, h defines the hidden layer of the current state, h_t-1 indicates the activations of the previous hidden layer state, and σ is the activation function. W and U are the learning weights of the network. In the remember neural network, some of the information is forgotten and some of the information is remembered using $\otimes$ gate, and the result is added to the final output using $\oplus$ gate. This separate neural network tries to distinguish what to remember and what to forget. We need to compute the information that we can learn, ie, a candidate addition to our long-term memory and its done using the ltm network. The save model uses $\otimes$ gate to act as a voting network to decide what should be kept internal and what should be released for the next iteration. The focus neural network is an attention mechanism where it filters the predictions that are not required instantly at the moment and uses $\otimes$ gate.

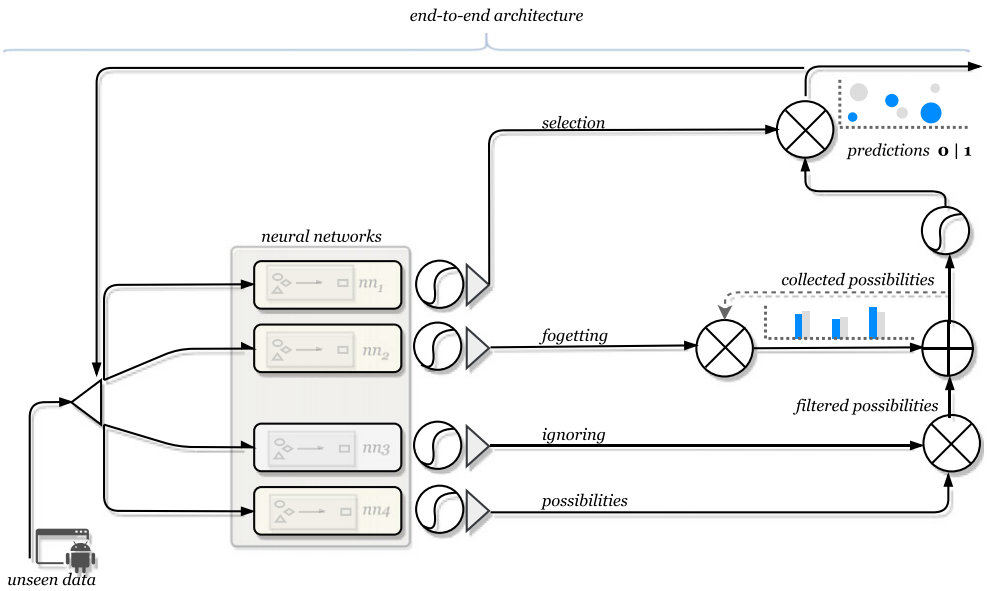

FIGURE 2 Long short-term memory architecture

Each hidden layer h for a given input x is calculated by a current weighted matrix on a hidden layer and addition to previous hidden layer values. Note that the main model of LSTM either learns character by character or sequence by sequence. The model is shown in Figure 2.

2.9.1 | Deep convolutional GAN

Note that the most widely used, adaptable, and standard model of GANs is that of deep convolutional GANs (DCGANs),[36] which deals unsupervised learning for datasets of images. For the sake of understanding, we discuss a brief introduction to DCGANs. Commonly, GANs are used for images (cf, Section 3). To modify the GANs to work for large images, convolutional networks are used. A convolution operator in both networks of GANs replaces the matrix multiplications in neural networks. Convolution usually changes the shape of the feature map as we move through the networks. When an image is an input to the model, it originally contains the width, height, and channels as its features. After passing through several convolutions and max pooling layers, we are left with very short and narrow feature maps. These maps are then fed to the final layer for the output. This operation is extended in GANs since we want to do the opposite for the generator model by increasing the feature map from small to a wide and tall image. For this operation, we need an architecture that can increase the width and height of feature maps at each layer, which is activated through transpose convolution. In the following section, we provide a brief literature review about how GANs are adopted.

3 | APPLICATIONS OF GANS

Before diving into the technical details of how GAN works, we should look at the most recent and popular variations explored using GANs.

- StackGAN: StackGAN[37] model is used to take input as a textual description of an input and to generate high-resolution output as an image matching that description. These generated images are unique and have never been seen before.
- Pix2Pix[38]: There are many GAN implementations developed by iGAN team in collaboration with Adobe to help mainly artists in their work. One of the applications is converting user-generated rough sketches made by pointing devices such as a mouse to a more possible real-life image. Another application is converting images from one domain to another, for example, translating building blueprints to complete finished building images.
- Nvidia GAN: Researchers at Nvidia[39] collaborated into GAN advancement by proposing a model that can, for example, convert an image of a day scene to an image of a night scene.
- CycleGAN[40]: A team at Berkeley presented an image-to-image translation unsupervised model. It can take images (or frames of a video) as a horse and can transform it into a zebra.

- Simulated training sets[41]: GANs can be used to take three-dimensional (3D)–rendered images of eyes and turn them into more realistic eye images. This can be used by computer vision models, which can take these realistic images to estimate the direction of eyesight.
- Imitation learning: GANs can also be used for imitation learning.[42] The actions taken by any reinforcement-learning agent can be taken as a kind of input data. This way, a GAN can learn to imitate the actions taken by a human expert.
- GAN in physics[43]: GAN can also be used to represent less visual data as well. For example, GANs are employed to predict the outcome of high-energy particle physics experiments. Instead of following Monte Carlo simulation, GAN leans to predict what is likely to occur in each simulation. It is helpful here in the sense of reducing the computational cost of simulating these physical experiments, saving million dollars' worth of supercomputer time.
- Semi-supervised Learning[44]: One of the main use of GAN is a classifier from unsupervised data as well along with label data to lower the amount of time used by deep learning models.

4 | RELATED WORK

In this section, we provide a brief overview of malware detection techniques for Android using conventional machine learning methods as well as recently discovered deep learning architectures. We also provide a comparison of generative models with our proposed model of GANs, which serves as our motivation to use them, and how a GAN-based technique tends to be a better choice as compared with the rest of the techniques. Figure 3 provides a brief visual representation of our related work.

4.1 | Comparison with conventional methods for malware detection

The Android malware analysis techniques can be broadly classified into the static analysis, dynamic analysis, and a combination of these two techniques.

Static analysis. The static analysis methods are the type of models that use static features, which are extracted from the app manually, for example, requested permissions and application programming interfaces (APIs) to detect a malicious app.

Arp et al[49] have proposed Drebin, a famous lightweight yet referenced as a de facto standard method for detecting Android malware. It enables identifying malicious applications directly on the smartphone of a user. Drebin performs a broad static analysis to gather as many features of an Android application as possible since the limited resources are required for monitoring applications at run time. Such extracted features are then turned into a joint vector for the task of malware identification by linear SVMs. Drebin outperformed several related approaches and detected 94% of the malware with few false alarms in their results.

Karbab et al[50] have proposed Cypider framework for static malware analysis. They present a set of techniques and tools to perform a systematic detection of smartphone malware by building an efficient and scalable similarity network

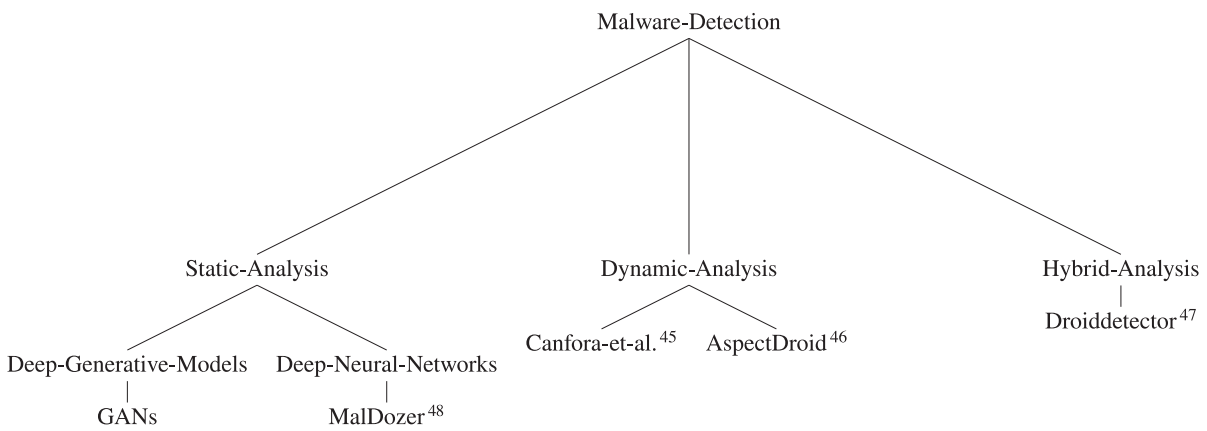

FIGURE 3 A brief taxonomy of malware detection. GAN, generative adversarial network

infrastructure of malicious apps. Their detection method mainly relies on a concept called malicious community, in which authors consider the instances that share common features for a given family. Using this assumption, multiple Android applications with different authors but similar and common features are most likely to be malicious. Cypider uses this assumption for the detection of all the variants of known malware families including *zero-day malware*. Cypider did not rely on signature-based patterns; rather, it was applied to community detection algorithms on the similarity network, ie, those that extract subgraphs are considered suspicious and are most likely malicious communities. Cypider showed clean results by detecting about 50% of the malware dataset in one iteration.

Mariconti et al[51] have presented a framework as MAMADROID, which is an Android malware detection system that counts on the behavior of an application. MAMADROID builds a behavioral model in the form of a Markov chain, by using the sequence of API calls made by an app and extracting features from the API calls in instructions to perform classification.

Dynamic analysis. The dynamic analysis methods aim for identifying behavioral signature or behavioral anomaly of the running app. The dynamic methods on the other hand offer limited scalability as they add additional cost in terms of processing and memory to the model.

Canfora et al[45] proposed an approach for capturing malicious behavior in terms of device resource consumption by using an exhaustive set of features. They described a procedure, as well as the corresponding practical setting, for extracting those features with the aim of maximizing their discriminative power. Finally, they described the results they obtained while experimenting on more than 2000 applications, on which the proposed approach results in an accuracy greater than 99%.

Ali-Gombe et al[46] presented an application-level system called AspectDroid, which is designed to investigate Android applications for possible unwelcome activities. AspectDroid is composed of application instrumentation, automated testing, and containment systems. AspectDroid weaves monitoring code into an existing application by using static bytecode instrumentation and provides data flow and sensitive API usage as well as dynamic instrumentation capabilities. The newly repackaged Android application is then executed either manually or via an automated testing module. They tested their system on 100 real malware families from the Drebin dataset.[49] The result of their analysis showed AspectDroid experienced approximately 1 MB of average total memory size overhead and an average increase of 5.9% in CPU usage.

However, the dynamic methods provide limited scalability as they face an additional cost in terms of processing and memory.[47]

Hybrid analysis. The hybrid analysis methods, on the other hand, combines both techniques to improve detection accuracy. However, the overhead of the hybrid approach is twofold during checking: offline check for static detection and online check when the application is in execution to cater the dynamic detection. This makes it very costly to carry such type of analysis and detection. Beyond this, we aim to conceive a solution that is computationally tractable. Yuan et al[47] implemented an online deep learning–based Android malware detection engine called DroidDetector, which can automatically detect whether an application is a malware or not. They thoroughly tested DroidDetector and performed an in-depth analysis with thousands of Android apps and the extracted features that deep learning essentially uses to characterize malware.

Some methods measure the similarity between the features of two similar malicious code assuming that malicious applications of the same family share similar features. Kim et al[52] presented a method of Android malware similarity and clustering. First, a process for extracting the control flow graph in an Android application is invented. By extracting the control flow graph, they form structural information of methods in an Android application, which they termed as "4-tuple." It is necessary to compare the matching process after they created the structural information extracted from the control flow graph. It was divided into two steps: "initial matching" and "second matching." They proceed with clustering using the aforementioned process. They used the DBSCAN algorithm for clustering based on previous studies. It was 65.8% average using the structural information of the result of the clustering.

Ali-Gombe et al[53] proposed OpSeq, which scores similarity as a function of normalized opcode sequences found in sensitive functional modules as well as the application's permission requests. This combination of structural and behavioral features resulted in a distinctive fingerprint for a malware sample, and it helped in improving proposed model's overall recall rate. The actual results show that OpSeq correctly detected known malware with an F-score of 98%.

Deep learning–based detection. The deep learning techniques are more acceptable than conventional machine learning techniques for Android malware detection such as in the work of Yuan et al.[47] Research on deep learning for Android malware detection is recently having more attention than before. As an example, in 2016, Hou et al[54] proposed a novel analysis method named Component Traversal. It can automatically execute the code methods of each given Android application as thoroughly as possible. They further constructed the weighted directed graphs based on the extracted Linux kernel

system calls and then applied a deep learning framework relying on the graph-based features for newly unknown Android malware detection*(zero-day)*. Promising results demonstrated that their proposed deep learning method outperformed other alternative Android malware detection techniques.

In the work of Karbab et al,[48] Maldozer automatically extracts and learns the malicious and benign patterns from the raw sequence of the app's API method calls to detect Android malware. MalDozer is evaluated on multiple Android malware datasets ranging from 1000 to 33 000 malware apps, and 38 000 benign apps. Another approach is Malgenome, which is recent but leverages a dataset; however, this approach is officially discontinued and no longer provides any support for malware detection.

Issues with existing methods: Remember that all these techniques either use manually crafted features and/or relies heavily on the expert's opinion. In addition, these models are not end-to-end models completely, meaning that creating and extracting features from the apps and labeling them either as malicious or benign are not accompanied by the model in a single go. Moreover, the datasets used in any of the technique are generated by the authors themselves to match the size and performance of their experiments. We counter this scenario by obtaining the third party or independently maintained datasets, which are live repositories and are considered to be the de facto standards. These are, arguably, the largest datasets available for malware detection until to date.

4.2 | Comparison with generative models

In this section, we provide the rationale of using GANs compared with not only the conventional machine learning algorithms but also compared with our employed model of GANs with other generative models and the improvement they provide over any other model.

Figure 4 provides a brief taxonomy of generative models that are closest to GANs in function and performance. Generative models could be either explicit or implicit density–based models. Explicit density–based models assume a prior and defined distribution about the data with the explicit parametric specification and are therefore termed as explicit density models.[56] Almost all machine learning models from naive to the state of the art use this form of density functions. This limits their ability to be applied to very large and unlabeled data.

Whereas, generative models such as GANs, does not make any assumptions about the data distribution beforehand and rather estimates the data distribution on their own; hence, they are called as implicit density models.[57] These models define a stochastic procedure to generate data and are often used as data simulators. We now provide the comparison of generative models in the following.

- Pixel RNN/CNNs[58] are models that are based on explicit density models. These models deploy an explicit chain rule to decompose the probability of an input x into the product of 1-d distributions as follows:

$$p_\theta(x) = \prod_{i=1}^{n} p_\theta(x_i | x_1 \dots x_{i-1}).$$

However, with this model, it is difficult to obtain a sample at a quick rate while maintaining a high quality of output samples, mainly due to the input traversal that is sequential and always depends on the previous input, which makes it further slower. In addition to this, PixelRNNs are yet to be applied for feature learning in general.

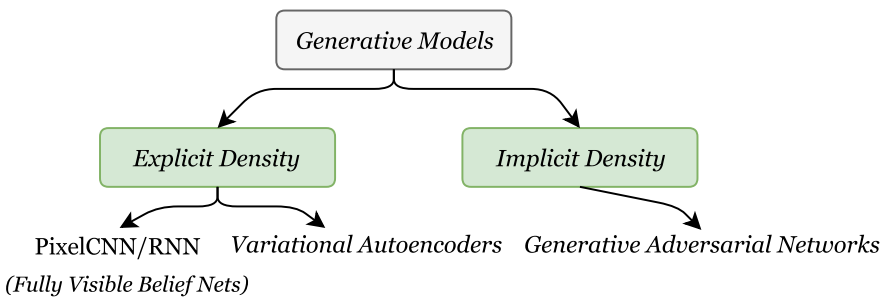

FIGURE 4 A brief taxonomy of generative models.[55] CNN, convolutional neural network GAN; generative adversarial network; RNN, recurrent neural network

- Variational autoencoders (VAEs)[59] are based on explicit density and are primitive generative models. VAEs define the density function with the inclusion of latent information z as follows:

$$p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz.$$

The architecture consists of two parts: encoder and decoder. The encoder maps the high-level features of a data distribution into a low-level representation of that data. This low-level representation of the input is called latent vector z. The decoder on the other hand takes this low-level representation of data and produces a high-level representation of the same. We can use an autoencoder to encode an input to a much smaller dimensional representation, which can store latent information about the input data distribution. However, in a vanilla autoencoder, the encoded vector can only be mapped to the corresponding input using a decoder. It certainly cannot be used to generate the output sample similar to the original input with some variability. This inability makes them incomparable to state-of-the-art GANs.

Details about GANs and how they handle issues faced by these aforementioned models can be seen in Section 5.

5 | GENERATIVE ADVERSARIAL NETWORKS

GANs are a kind of generative models that can generate the complete input, eg, an image, or a character sequence in one shot. It has two main parts, namely, a generator and a discriminator, and makes use of minimax game theory.[60] The generator G takes a random vector z of the form shown in Figure 5 as an input. This is usually termed as noise that runs through differential functions in each iteration such that the output of the generator network is as close as possible to a real input. These input vectors are made by looking at a fair amount of real input data distributions. However, for a generator to be generating a real looking sample, it has to be trained differently than the usual supervised models. Since there are no labels attached to the input samples or training data, the generator has to produce more look-a-like samples from the same probability distributions as that of real input samples. The generator is then optimized with a specific end goal to build the likelihood of the created information being weighted higher. Most generative models get to this process by some kind of approximation technique, eg, adaptive moment estimation (ADAM)'s optimizer. In GANs, an additional approximation is also used from the second model Discriminator D, where it learns to guide the generator model in parallel through sharing its loss as shown in Figure 6. The discriminator D is only a regular neural network that takes real input samples from two sources: one half from the real input dataset and the other half from the sample generated from the generator's output as real or generated $G(z)$. The job of the discriminator is to discriminate and assign the probability $P(x)$ near 0 for real samples and 1 for fake samples. Simply, this process involves the generator trying to generate samples that are true copy of the real input of samples and can be recognized as real by the discriminator model. In other words, the generator is forced over time to fool the discriminator by generating samples near to real data. Therefore, it generates them by using random noise and passes them to the discriminator; meanwhile, the generator backpropagates and tries to adopt distribution of the real samples. Remember that the real data or samples are never shared with the generator. This process continues until the discriminator is fully aware of the fake samples generated by the generator and can successfully distinguish between a real and a fake input sample. In fact, the discriminator is optimized in such a way that it has a likelihood of producing low probability of generated data and a higher probability of real data. In addition, it is hard to tackle the convergence in GANs, since they are different from usual neural networks, which consists of a single player, whereas GANs are based on two-player minimax game. Finding the suitable point, in this case, the saddle point (a value v where D is max, and G is min), to stop the training and seek convergence is an issue due to the nonconvexity of the discriminator and the generator. Finally, however, for comparing the similarity between probability distributions of both neural networks, we use Jensen-Shannon divergence[61] bounded by [0,1].

5.1 | Implementation of GAN for training

It is very important to modify the design of GAN and the overall architecture to make the network work well in all scenarios. If the task is as simple as generating 28×28 pixel images of MNIST handwritten dataset using GANs, then the

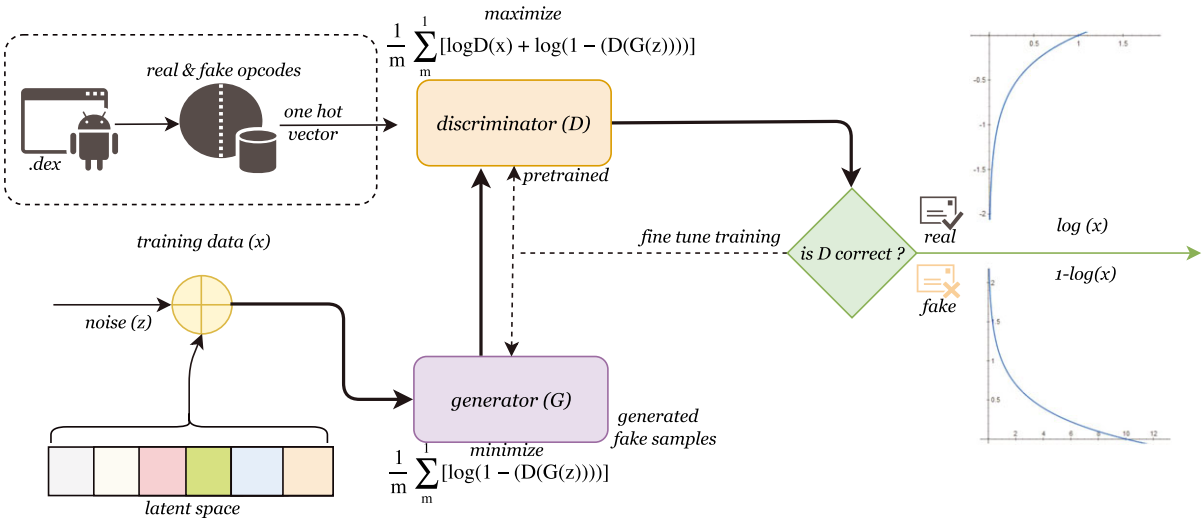

FIGURE 5 Block diagram of long short-term memory generative adversarial network architecture used in our experiments

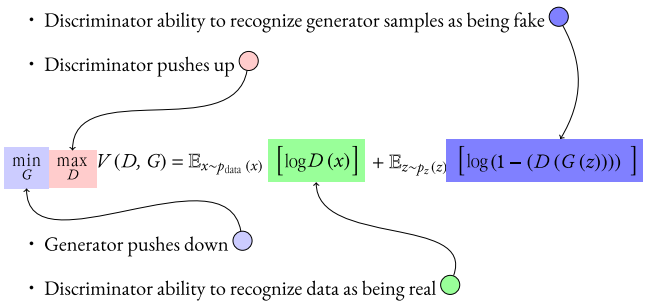

FIGURE 6 Generative adversarial network

fully connected architecture as that of DCGAN is used, which is introduced previously (cf, Section 2.9.1). In DCGAN, fully connected architecture of layers performs matrix multiplications by their weight matrices, and most importantly, hidden layers are used once at least between two networks to ensure that they can approximate any probability distribution if hidden units are given.

This is the main idea behind DCGAN architecture, where we increase the feature maps of the generator input to the size of real input images for the discriminator. Most of the research and implementations around GANs is following the same architecture as that of DCGANs. However, since the images are fed as either two-dimensional (2D) or 3D, GANs are optimized to process such input datasets. As opposed to this, sequences, which are represented as one-dimensional (1D), cannot be mapped directly to higher dimension input such as images. Therefore, GANs need to be modified to accept the input vector in 1D and to still process them as usual input space, akin to 2D or 3D. In our case, the input is binary opcode sequences extracted from Android Package Kit (apk) binaries, represented as 1D, which the authors were unable to feed to the GANs initially. This is the rudimentary reason for using a variant of GAN; in addition, GANs are highly suitable for unlabeled data[62] to be trained and tested, as discussed in detail in Section 7. In static malware detection, it is assumed that the nature of the malware is unknown but the malicious app will result in a different behavior from that normally seen in the system.

6 | DATASET

We now discuss the sketch of our input dataset, which is extracted from the apk binaries of Android.

For all our experiments, we have used three state-of-the-art datasets available for the analysis and detection of Android malware. Note that we augmented the bytecode dataset extracted from the dalvik executable (.dex)[63] in the apk collection in these datasets and stored them in an unlabeled repository. Thus, we do not consider any code other than present inside a .dex file nor do we look for the obfuscated code.

The first and most commonly used dataset is that of Drebin[49] dataset of real and malware Android applications. This dataset contains the samples collected over a time-lapse of more than two years, ie, from August 2010 to October 2012. This dataset contains 123 453 benign and 5560 malware apks collected from the official Google Play Store, the Russian and Chinese markets, etc. To be sure, all the samples are sent individually to the VirusTotal service, and the behavior of each sample is tested through ten antiviruses such as Kaspersky, Sophos, and AVG to name a few. It provides an API service to run the apks through a large number of antivirus vendors. It detects and returns the report of each antivirus about the apk sample of being malicious or benign. Drebin uses 545 334 features to distinguish benign and malicious apps. Out of these features, 315 Android API calls are considered suspicious. Drebin authors have chosen these calls out of the 20 000+ from the Android API: API calls for accessing sensitive data, such as *getDeviceId()* and *getSubscriberId()*, and API calls for communicating over the network, for example, *setWifiEnabled()* and *execHttpRequest()*, just to name a few. Additionally, all the major families encompassed by this dataset are attributed in Table 1.

The second dataset that we have used to extend our bytecode sequences is the Android Malware Dataset (AMD).[64] This is a vast repository of Android apks collected over a long duration from 2010 to 2016 and has a wide variety of 135 composite families of benign and malware apps. Due to computational infeasibility, we used only 60 GB of this dataset. Overall, this dataset has 24 533 apks with benign apps contributing to the 47% of the total dataset volume. Major families found in this dataset are attributed in Table 2. To balance and make our contribution novel, we added the apks

TABLE 1 Attribution for Drebin dataset

S.No	Family	# of Samples	S.No	Family	# of Samples
1	FakeInstaller	925	11	adrd	91
2	DroidKungFu	667	12	DroidDream	70
3	Plankton	625	13	LinuxLotoor	81
4	Opfake	613	14	GoldDream	69
5	GingerMaster	339	15	MobileTx	69
6	BaseBridge	330	16	FakeRun	61
7	Iconosys	152	17	SendPay	59
8	Kmin	147	18	Gappusin	58
9	FakeDoc	132	19	Imlog	43
10	Geinimi	92	20	SMSreg	41

TABLE 2 Attribution of major families in the Android Malware Dataset

Number	Family	Samples	Number	Family	Samples	Number	Family	Samples
1	Airpush	7843	25	GoldDream	53	49	Roop	48
2	AndroRAT	46	26	Gorpo	36	50	RuMMS	402
3	Andup	45	27	Gumen	145	51	SimpleLocker	173
4	Aples	21	28	Jisut	560	52	SlemBunk	174
5	BankBot	648	29	Kemoge	15	53	SmsKey	165
6	Bankun	70	30	Koler	69	54	SmsZombie	9
7	Boqx	215	31	Ksapp	36	55	Spambot	15
8	Boxer	44	32	Kuguo	1199	56	SpyBubble	10
9	Cova	17	33	Kyview	175	57	Stealer	25
10	Dowgin	3385	34	Leech	128	58	Steek	12
11	DroidKungFu	546	35	Lnk	5	59	Svpeng	13
12	Erop	46	36	Lotoor	329	60	Tesbo	5
13	FakeAngry	10	37	Mecor	1820	61	Triada	210
14	FakeAV	5	38	Minimob	203	62	Univert	10
15	FakeDoc	21	39	Mmarketpay	14	63	UpdtKiller	24
16	FakeInst	2172	40	MobileTX	17	64	Utchi	12
17	FakePlayer	21	41	Mseg	235	65	Vidro	23
18	FakeTimer	12	42	Mtk	67	66	VikingHorde	7
19	FakeUpdates	5	43	Nandrobox	76	67	Vmvol	13
20	Finspy	9	44	Obad	9	68	Winge	19
21	Fjcon	16	45	Ogel	6	69	Youmi	1301
22	Fobus	4	46	Opfake	10	70	Zitmo	24
23	Fusob	1277	47	Penetho	18	71	Ztorg	20
24	GingerMaster	128	48	Ramnit	8			

from the well-known VirusShare[65] repository, which is the third source of our data. It is a verified and ongoing live malware repository and contains 24 317 apks. This dataset is not yet attributed. This is an ongoing project that can be seen here.[66]

We note the behavior in terms of the bytecode of an apk, presently opcode sequences of binary patterns with a length of 16 bits. The primary thought is to make up a different database of bytecode for each apk in question. Having versatile and large patterns on how each executable is configured, patched, and used, we believe that this database will provide a reasonable ground of training and testing for most of the frameworks in the future. When this database is developed for all the given apks, the database can be utilized to screen the apk's ongoing behavior. The generator learns these binary sequence patterns and passes them to the discriminator for evaluation through a mechanism of learning nearby bytecode sequences through windows of varied sizes slide over them, as shown in Figure 7. The sequence of binary patterns in the bytecode, in the training set, which the system is trained on, and the unusual sequences indicate anomalies in the running apk. Notice that we have adopted and extended this approach devised in a seminal work[67] for malware detection, which has shown a lot of adaptations over the years.[68-70]

These three datasets are intermixed and a resulting volume of 2 million was obtained. Input data from apks *(.dex)* is stored in a text file, which is fed to a function that converts it into a one-hot numpy tensor, which has a shape shown in Figure 8.

The data is composed of 127 188 000 000 different integer words between 0 and 1 of length 16. The second dimension passed to *X_actual* variable represents the number of rows in a tensor, whereas the third dimension refers to the

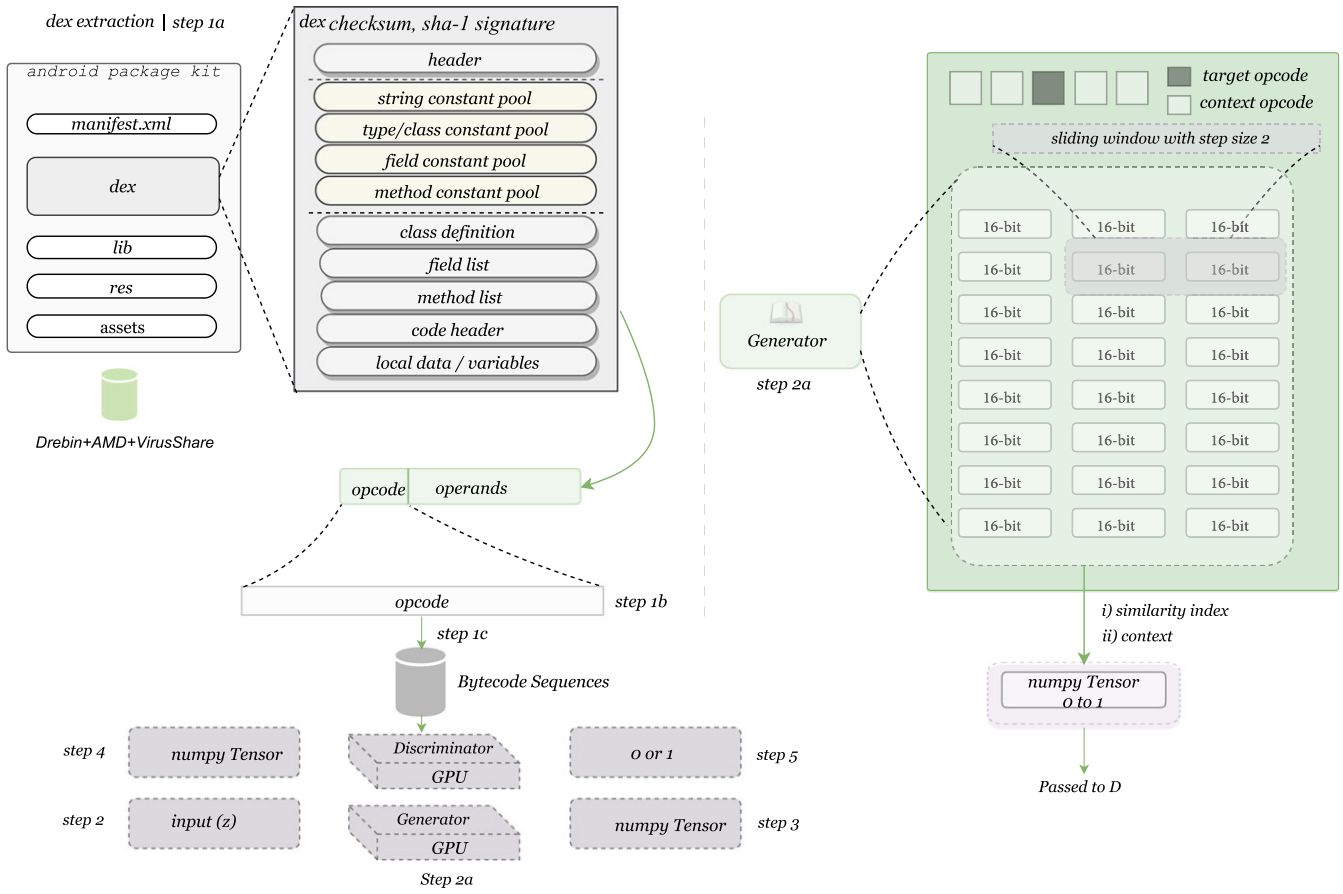

FIGURE 7 A sketch of how generator learns bytecode sequences. GPU, graphics processing unit

```
X_actual.shape

(127,188,000,000, 16, 3)
```

FIGURE 8 Dimensions of numpy tensor

number of columns in each row. For a network to be trained on this data, which is large on an order of magnitude have to be represented in some numerical way and fed further for the training process. There are many ways to convert sequential data into a numerical form like Word2Vec[71] and Glove.[72] However, we employed none of these methods, and for our experiments, we are reading data line by line and converting each line into a numpy tensor. Each word read by a line is treated character by character and is represented by a row, where each row contains only 1 bit *on* and all other bits are 0 or *off*. This representation is referred to as one-hot encoding. For example, in the given sequence *xi*(0000000000000001), the 16th bit is 1 *on* and the remaining bits are 0's *off*. For this sequence to be represented as a numpy tensor, the number of row in the tensor will be 16 (which is the length of the opcode sequence), and each row has a format like ['\x00','0','1']. The first column in this row represents a space character. Since the first bit in the given sequence *xi* is 0, it will be represented as [0, 1, 0], while the 16th bit is 1; therefore, the corresponding tensor row is represented as [0, 0, 1]. This way, each 1D word and opcode in our case get into matrix form, which is now in a shape to be fed directly to the GANs for training and testing. The matrices representing each opcode sequence in tensor form are as shown in Figure 9.

This tensor is received by the *X_actual* variable, which is an input to the discriminator as shown in Figure 9. This tensor is then fed to the discriminator as real data. Now, the discriminator tries to minimize the probability to reduce the loss. Once the discriminator is trained, it outputs binary values using the MatrixToInteger function as shown in Algorithm 2.

Rationale of these datasets. Note that these datasets are the latest available datasets, which are live and officially maintained by their contributors. In addition, these are the largest Android executable repositories and are used as de facto standards for mobile malware detection. One can briefly skim through Tables 1 and 2 to comprehend about the major attacks and malicious families found in these datasets for the Android platform.

Note that our generator *G* is using the context window slid over the samples to know the sequence patterns, as shown in Figures 7 and 10, which can apprehend the metamorphic and dynamic code successfully. This method is already proven to be successful on previous studies.[73,74]

Our created dataset is in a sense unique and novel as our contribution is the largest and updated bytecode repository *freely* available. To the best of our knowledge, these three datasets are yet to be used in conjunction for GAN-based malware detection.

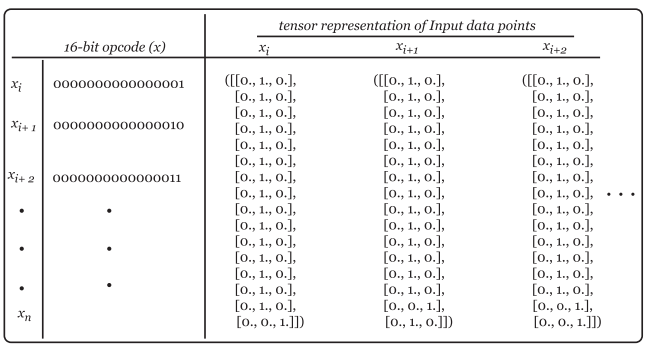

FIGURE 9 Instances of opcode sequences into tensor vector

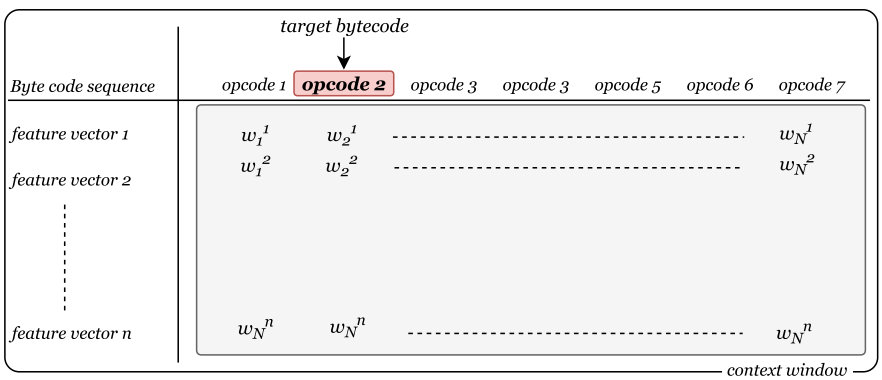

FIGURE 10 An example of how bytecode is detected

7 | PROPOSED ARCHITECTURE

7.1 | LSTM-GANs

Now, we discuss the actual model behind each of the discriminator and generator in GANs. The neural architecture we are using here for implementing GAN is by using LSTM networks. So far, the GANs we see are variants of DCGAN, and almost all of the times they are being trained on image data specifically, as explained in Section 5. However, we wanted to feed the opcode-based binary sequential data to the GANs, and training a simple CNN might be easy but configuring CNN with GANs for unsupervised learning is computationally an infeasible task. Since CNNs are very handy at dealing with high dimensional data, but dealing sequences with them is not a very common approach these days. Therefore, our obvious choice was to choose LSTMs to learn on a large and unlabeled dataset (cf, Section 6). Another reason is that our input data is in 1D shape, which is simply different from the 2D and 3D image inputs. The LSTM architecture is thoroughly discussed in Section 2.9.

Note that, inside the discriminator and generator, we have incorporated LSTM as deep learning architecture and reiterate that the discriminator and generator are traditional neural networks, which are best suited for sequential data. The very first task is initializing the parameters such as hidden layer size in the generator and discriminator, the number of epochs for overall GAN, and for each network training, the initial learning rate, the batch size, and the pointer to a dataset.

For the generator, an input is always a random value in a vector equal to the size of the actual dataset. This is fed to the generator, which produces tensor equal to the size and dimensions of *X_actual* so that it can be processed and forwarded to the discriminator.

As described in Figure 5, at each timestamp of the LSTM model, the generator takes the latent input that will train itself based on the loss calculated by the discriminator to maximize the probability of wrong labels. This way, it tries to produce samples as close to the original binary opcodes using a scheme that we can see in Algorithm 1. Once done, the generator generates a tensor, which is concatenated to real input samples provided to the discriminator and passes it to the discriminator so that it can classify the output as real (0) and fake (1). This scheme is depicted in Algorithm 2.

The discriminator at the end of learning generates gradients and passes them to the generator so that the model uses those to update the weights. After that, a dictionary of unique characters is created in a dataset with the key as index and value as a character. In our case, it is 0 and 1. The GAN class object is initiated with parameters, and then the train function is called where all the parameters are passed for actual learning. The GAN train function starts with creating first the fake data for the generator network equal to the size of the actual data.

It is first fed to the generator model to generate some text without any training, and the result is concatenated with input to the discriminator training model. The accuracy of discriminator before training the generator is calculated. Subsequently, the generator is fed with the fake data (z) (using Gauss distribution) again to start training along with loss (gradients) provided by the discriminator under the given number of epochs. The generator model is run for feeding forward its output so that it is compared with the discriminator again for calculating the accuracy at each epoch.

Figures 7 and 10 have shown a brief scheme of how a target bytecode is evaluated and its nearby neighbors are detected in a given sequence. When the context window is slid over the bytecode sequence, it learns which opcode follows which in the context window. Whenever an opcode occurs more than once, it can be followed by numerous different opcodes. These are all learned. The test data sequences or sequences of zero-day malware are also passed through context window determining the sequence of bytecodes that differs from the previously recorded sequences. If the sequence mismatch at one or more locations, it would help evaluate and characterize the behavior of the apk. Note that, in our case, we are dealing with a sequence-based input dataset so LSTMs best suited us. Authors did try other models like CNNs but our experiments did not go well and LSTMs yielded the best results for our experiments.

8 | EXPERIMENTAL SETUP

Our dataset nomenclature was discussed in Section 6. Now we turn towards the brief outlines of our experimental setup.

Each application's data is converted to the one-hot representation of the opcodes extracted through static analysis from the bytecode directly. Note that we have evaluated the predictions of our deep learning models by splitting

our dataset into training and testing sets with a ratio of 60% and 40%, respectively. We also performed standard ten-fold cross-validation for each of the training and testing phases. First, we provide the list of tools and associated libraries, which were used in a setup for the analysis environment. Next, we share the results obtained from these experiments.

8.1 | Environment and tools

To have an interactive and intuitive setup, we used several data science packages along with Python. It is important to note that the architecture that has input mechanism, uses one-hot encoding, and reshapes inputs to tensors and feed them to the generator and adversarial parts of GAN is simply not imported from the built-in models; rather, the architecture adopted is fully written consuming the famous numpy package and support.

We now enlist the libraries and packages, which are broadly used in our training, testing, and evaluation phases of our research.

1. *numpy*[75] and *pandas*[76]: These are essential packages used in Python for mathematical computing and data sciences.
2. *h5py*: h5py[77] acts as a binder between Python and binary data in HDF5 format. This can immensely offload the burden from the user to the machine to load massive datasets being prepared with numpy, and this utilizes the machine to shift the data to or off the system memory automatically via traditional batch data processing.
3. *Matplotlib*[78] and *seaborn*[79]: These libraries are heavily used for the visualization and plotting of the obtained results. Both the libraries come with an awesome user support.
4. *Sklearn*[80]: This is the tool behind generating, evaluating, and analyzing the confusion matrix and related metrics required in calculating results against our architecture.
5. *Natural Language Toolkit (NLTK)*: NLTK[81] is an open-source and free symbolic and statistical linguistics tool based on Python. Our analysis with a Lexical dispersion plot was carried out amicably with this toolkit.

Considering a very large size of our input space, we conducted several experiments with varied input amounts of data. Initially, the daily life PC sufficed for our experiments, but as soon as the input began to become large, it was simply not possible to use ordinary CPU-based system for our 24/7 executions. As an alternative, we switched our focus to a much rugged graphics processing unit (GPU)–based machine with a handsome 32 GB built-in system memory, a dedicated 8-core processor clocking at 3.5 GHz modeled as Core i7-4770k (4th generation) from Intel.

It is evident that once the neural networks are trained with various hyperparameters on these handsome machines, in addition to the availability of stable and lightweight implementations of neural networks across all prevailing platforms, it is flat to configure these pretrained networks on the Android hosted platform even with bare minimum hardware configurations.

9 | TRAINING GENERATIVE ADVERSARIAL NETWORK

9.1 | Generator

The generator first generates the expected output to the discriminator; then, it collects gradient for its own weight learning by minimizing the $\mathbb{E}_{z\sim pz}(z)[\log(1 - D(G(z)))]$. After computing gradient for a particular batch, it updates its learning rates as well as the associated weights. The generator is then run through feedforward and backpropagation for the number of epochs provided.

It has been trained so that the discriminator at one point starts accepting the real data as well as data generated by the generator as a real one. The network details for the generator are summarized as follows:

$$\text{z} \rightarrow \text{randn(num_examples,generator.input_size)} \rightarrow \text{generate_tensor(num_examples,sequence_length,char_list)} \rightarrow \text{G_forward_prop(X,sequence_length)}.$$

This scheme is described in Algorithm 1.

Algorithm 1 Generator

```
function TRAIN(self, X, seq_len, discr, num_epochs, data_index, initial_learning_Rate, grad_multiplier, batch_size)
    num_examples ← X.shape[0]
    ms ← 0

    for i ∈ Range(num_epochs) do

        gen_output ← self.generate_tensor(seq_len, num_examples, X)
        Y_exp ← np.zeros((num_examples, seq_len, 2))
        Y_exp[:, −1, 1 − data_index] ← 1

        discr_grads ← discr.lstm.BPTT(gen_output, Y_exp, discr_dloss, Return_list ← True)
        discr_grad_x ← np.array([grad[0].dLdxforgradindiscr_grads])
        discr_grads_x ← disc_grads_x.swapaxes(0, 1)
        genr_dloss ← lambdah, y : y

        if batch_size is None then
            grad ← self.lstm.BPTT(X, discr_grads_x, genr_dloss, seq_len)

        else
            batch_indices ← np.random.choice(np.arange(0, num_examples), batch_size)
            input ← X[batch_indices]
            exp_outp ← discr_grads_x[batch_indices]
            grad ← self.lstm.BPTT(inpt, exp_outp, genr_dloss, seq_len)
        end if

        magnitude ← sum([gl.magnitude_theta()forglingrad])
        ms ← (1 − grad_multiplier) ∗ ms + grad_multiplier ∗ magnitude
        lr ← initial_lr/math.sqrt(ms)

        self.lstm.update_theta(grad, lr)
    end for
end function
```

9.2 | Adversarial network

The adversarial network (discriminator) class has many functions, and one of them is training the discriminator model on real + fake data by maximizing $\mathbb{E}_{x\sim \text{pdata}}(x)[\log D(x)]$. The discriminator trains the classifier based on two classes: 0 for real data and 1 for fake data. After the learning process, the model is tested via passing both generator and discriminator data to see the accuracy of predictions. The network details for the discriminator are given as follows:

x → X_actual → D_train_RMS(X_actual,genr_output) → concatenate(X_actual, genr_output) → LSTM.RMSprop(X,Y,loss,dloss,epochs,initial_lr,batch_size) → feed_forward(X,sequence_length) → BPTT(X,Y,dloss).

The steps are clearly mentioned in Algorithm 2.

Algorithm 2 Discriminator

```
function TRAIN(X1, X2, num_epochs, initial_learning_rate, grad_multiplier, batch_size)

    # wrangle input and output data
    X ← np.concatenate((X1, X2))

    Y ← np.zeros((X.shape[0], X.shape[1], 2))
    Y[: X1.shape[0], −1, 0] ← 1
    Y[X1.shape[0] :, −1, 1] ← 1

    self.lstm.RMSprop(X, Y, loss, dloss, num_epochs, initial_lr, grad_multiplier, batch_size, print_progress ←
print_progress)
end function
```

9.3 | Matrix to integer

The tensor generated by a generator model is first passed through a function called matrix_to_string function, where the tensor is passed to convert a character embedding to a string. This string is used to display the output learned by the generator. The steps are shown in Algorithm 3.

9.4 | Hyperparameters

There are certain hyperparameters we have to take care of so that the learning process is able to achieve its objectives. Some of them are discussed in the following. Note that we used random search[30] optimization for hyperparameters in our experiments. First, the learning rate plays a vital role in training any deep neural networks. The initial value should be set according to the type and size of the dataset and to the number of epochs.

Usually, the values near 0 are more preferred, eg, 0.1 or 0.001, because they are more helpful in learning the large datasets. This way, the gradients will get to the bottom of the slope slowly and with better learning. A number near to 1, eg, 0.9 will help learn the errors much earlier but in large epochs; it brings the gradient down to zero very early, and there will be a point where there will be hardly any learning left. In our case, we choose to set the learning rate to be 0.1.

Algorithm 3 MatrixToInteger

```
procedure MATRIXTOINTEGER(tensor)
    answer ← εε
    for all c ∈ tensor do
        ind ← np.argmax(c)
        answer ← answer + self.char_list[ind]
        RETURN answer
    end for
end procedure
```

Similarly, another choice to set from is the number of *hidden layers*. The more the hidden layers in the network, the deeper the model gets for learning a large dataset. Nevertheless, at the same time, the more complex it will become in representation. We usually prefer a minimum of two to three hidden layers in following a good standard without any trouble.

In addition, the number of neurons in each hidden layer can be different or the same. Commonly, it is a process of hit and trial method to see in which settings our model performs well. We tried to train and test our models with four and five hidden layers, and this is the optimum number where we achieved our best results. *Epochs* is the number of times a dataset will be iterated through the model in training mode. This is usually preferred to be a large number but

TABLE 3 Hyperparameters

Epoch	D-Epoch	G-Epoch	Time-elapsed	Loss	Accuracy	Learning rate	Batch size
0	20	20	54.53 sec	0.412	0.8	0.1	64
1	20	20	60.56 sec	0.0228	0.9	0.1	64
0	20	20	53.53 sec	0.2	0.75	0.1	128
1	20	20	59.56 sec	0.0228	0.89	0.1	128

when the data points tend to get larger as in our case, then it needs to be regularized. This value is highly dependent and can be increased with the support of GPU. The number and rationale behind selecting the epochs is discussed in Section 9.5. During the training phase, we varied the number of epochs for the generator and discriminator model as well as the nested epochs inside the main epoch iterations. We ran our model for a very large number of iterations but it did not do well. To improve the accuracy and avoid overfitting, we carefully used early stopping which is just another form of regularization. *Batch size* is the size of the batch, which also plays a vital role in learning a dataset. The larger the batch size is, the larger the data will be stored in the memory at a time as a chunk. This way, the CPU can avoid fetching individual data points from the main memory. Batch size is chosen in the exponential of 2, ie, 64, 128, 256, 512, etc. Usually, the 64 and 128 are preferred more in deep neural networks and that is why we are using these values as our final batch size. For a quick review training hyper-parameters are summarized in Table 3.

9.5 | Training

The training process of LSTM-GAN involves several steps through which one epoch is completed for both the generator and discriminator. Note that LSTMs are responsible for devising a mapping from an input (opcodes in our case) sequences $x = (xi \ldots xn)$ to an output sequence $y = (yi \ldots yn)$ via computing the activations in network.

LSTMs are composed of three basic layers: (1) an input layer, (2) an LSTM layer having recurrence, and (3) a standard output layer. The input layer precedes the recurrent layer and is directly connected to it. The LSTM layer has cell connections from the output to input units, and all the gates in the architecture, namely, input, output, and forget (cf, Section 2.9).

The output layer follows the recurrent layer and receives the input from the cell output unit of the previous layer. As discussed in Section 2.9, all the parameters W in the LSTM network can be computed as follows:

$$\mathcal{W} = \mathcal{M}_c * \mathcal{M}_c * 4 + \mathcal{M}_i * \mathcal{M}_c * 4 + \mathcal{M}_c * \mathcal{M}_o + 3.$$

In this equation, $\mathcal{M}_c$ refers to the number of memory cells (blocks), $\mathcal{M}_i$ refers to the number of units for input, and $\mathcal{M}_o$ refers to the number of output units. Since we perform optimization including stochastic gradient descent (SGD),[25] it is complexity for learning each weights and time step for LSTM network is *O(1)*, and per time step, it settles to *O(W)*.[82] Remember that learning for LSTM models tends to be computationally expensive if the processing involves several output units and multiple memory cell for saving contextual information over time.

The training process can be formulated as the information associated with the input is y and associated features are called as x. In this way, the expression $p\ (y \mid x)$ can be elaborated as "the probability of y given the x," which in our case can be mapped as "the probability that an input apk is fake given the opcodes it contains." For this process to work, the very first step is providing the dataset in numpy-tensor form along with the hyperparameters to GAN, where both the networks run separately.

The second step involves generating random input equal to the size of actual dataset and is feedforwarded through the generator network without training in the current epoch for producing output as a tensor. This tensor acts like fake data for the discriminator and is concatenated with the real dataset; additionally, their batches are maintained apart from each other. Then, the discriminator is iterated through training data to teach the discriminator what is real and what is fake.

Once the gradients of discriminator are learned, the generator model to update its weights for the current batch uses them. These steps are provided in the algorithm used in the work of Goodfellow et al,[20] as shown in Equations (6) and (7) of Algorithm 4. This process continues until the generator is able to generate samples as close as possible to the real data and the discriminator is able to judge them as belonging to either class, that is, real or fake.

We trained the GANs for 1000 epochs with batches of 64 and 128, which yielded the best results for us. In our experimentation, we found that normalizing the inputs to the range of 1 and -1 is a good practice. Note that GANs may get

unstable with sparse gradients. It is common practice to use the optimization protocol, eg, ADAM[83] optimizer, for both the generator and discriminator. However, we found that it is more useful to apply a combination of optimization techniques for both the generator and discriminator. We achieved far better results when we adopted the SGD for discriminator and ADAM for generator. Finally, we have noticed and recommend two advices for training generators: (1) Applying a proper dropout ratio increases the ability of the generator, and (2) it is better to apply proper noise(z) function and to sample from Gauss distribution, rather than just sampling some linear noise.

9.6 | Optimization

Optimization in GAN is different than other machine learning model since we have to deal with two neural network models and two different problems: (1) The generator is fed with poor quality feedback if the discriminator performs badly, and the generator is then devoid of the actual representation of the real samples, and (2) if the discriminator performs really well, the training process becomes extremely slow since the loss tends to approach to '0,' which might eventually stall the learning process.

Algorithm 4 Training GANs

for no. of training iterations **do**
  for each step k **do**
    description Sample minibatch { z (1) , . . . , z (m) } from prob-G (z)
    Sample minibatch { x (1) , . . . , x (m) } from prob-D (data) (x)
    Update D by ascending its stochastic gradient:

$$\nabla\Theta_d \frac{1}{m}\sum_{i=1}^{m}[logD(x^{(i)})] + [log(1 - D(G(z^{(i)}))] \tag{6}$$

  end for
  Sample minibatch { z (1) , . . . , z (m) } from prob-G (z)
  Update G by descending its stochastic gradient:

$$\nabla\Theta_d \frac{1}{m}\sum_{i=1}^{m} log(1 - D(G(z^{(i)})) \tag{7}$$

end for

Following the research of Indico and Facebook[36] for DCGAN models, the good choice for optimization is ADAM's optimizer. The discriminator and generator loss can be calculated by the following representation:

```
Dloss=crossEntropy(logits,labels)
Gloss=crossEntropy(logits, flippedLabels)
```

Here *flippedLabels* means that label 1 will be treated as zero and vice versa. This helps the generator to maximize the log probability of the wrong labels.

For discriminator, the loss will be equal to the sum of loss for real and fake data since both are passed together with 0 labels as real and 1 for fake data. The end goal is to make the discriminator output the label as 0 if the model thinks that the data belongs to the real dataset and 1 for fake generated data.

On the other hand, the generator receives Y object from the discriminator object and swaps the labels among 0 and 1 so that the generator can fool the discriminator for making fake data look like real. It then computes the gradients for generator weights to update and uses them to generate new output in the current iteration.

The optimization will run until a saddle point is reached where the discriminator tries to maximize and the generator tries to minimize the probability. This particular game can be measured by *Nash Equilibrium*[22] and can be shown as follows:

$$P_{\text{data}}(x) = P_{\text{gen}}(x)\forall x$$
$$D(x) = 1/2\forall x$$

where $P_{\text{data}}(x)$ is the probability density of the actual data, $P_{\text{gen}}(x)$ is the probability density of the generated samples, and $D(x)$ is the one half of the discriminator output every time.

10 | RESULTS AND DISCUSSION

We now discuss the evaluation criteria for our models. Commonly, binary classification is evaluated through the confusion matrix, which is divided into four classes: True positive (TP), which is the data that are classified as true and are actually true; true negative (TN), which is the data that are classified as false and are actually false; false negative (FN), which is the data that are classified as false but are actually true; and false positive (FP), which is the data that are classified as true but are actually false.

Recall is the ratio of correctly predicted positive observations with respect to all observations in actual class. It can be defined as

$$\text{Recall} = \text{TP}/T = \text{TP}/(\text{TP} + \text{FN})$$

These metrics are usually endorsed through F1 score, which is the harmonic mean of recall and precision, and it is specified as

$$\text{F1Score} = 2 * (\text{Recall} \ * \ \text{Precision})/(\text{Recall} + \text{Precision}).$$

Similarly, the classifier's ability to generalize can be measured by evaluating its accuracy. Accuracy is the ratio of correctly predicted observation to the total observations. In our scenario, the accuracy defines how exactly GANs identify the real and fake data using the trained model. The higher the accuracy, the better. Accuracy can be calculated by

$$\text{Accuracy} = \text{TP} + \text{TN}/\text{TP} + \text{FP} + \text{FN} + \text{TN}.$$

Precision is the ratio of correctly predicted positive observations to the total predicted positive observations. Precision can be indicated as follows:

$$\text{Precision} = \text{TP}/P = \text{TP}/(\text{TP} + \text{TP})$$

Out of all the positives, determining how many are identified as positive is known as the true positive rate, while the false positive rate shows out of all the negatives how many are identified as negative. They are determined as

$$\text{True Positive Rate}(\text{TPR}) = \text{TP}/\text{TP} + \text{FN}$$
$$\text{False Positive Rate}(\text{FPR}) = \text{FP}/\text{FP} + \text{TN}.$$

The receiver operating characteristic (ROC) expresses the model ability to predict regardless of your model threshold, while AUC is the area calculated under ROC curves using an integral.

The support metric allows the number of occurrences for each label in the true class labels. In our case, it is always equal to 0s and 1s as shown in Table 4, which exhibits that the dataset is balanced.

Following the definition of our assessment criteria, we now give the correlation of our research contribution to the present-day techniques.

10.1 | Comparison with existing techniques

The methodology we adopted to implement LSTM-GANs to detect the malware on Android platform was discussed in detail in Section 7. We have reverse-engineered the Android apks and extracted the opcodes from them, which were fed to LSTM-GANs as numpy tensors for further analysis. It is important to mention that we experimented with wide-ranging hyperparameters and came upon with nominal to state-of-the-art outcomes.

Deep generative models, such as GANs, tend to generalize comparatively better to unseen data. The implicit density functions in GANs stochastically generate data and learn the features in an unsupervised manner, which suffices for the

TABLE 4 Evaluation metrics

	Precision	Recall	F1 score	Support
Class 0	1.00	0.98	0.99	127 188 000 000
Class 1	0.989	1.00	0.98	127 188 000 000
Avg/total	0.995	0.995	0.99	254 376 000 000

analogy of identifying zero-day malware, ie, malware that is yet not seen and to be fixed, and its properties and features are unknown to the detection systems. Remember that zero-day malware is similar to the test data that is also not seen by the detection system, yet the system is able to characterize it as malicious or benign. Quite a few techniques using static malware detection techniques have already proven to be successful against zero-day malware detection.[9,50]

We reiterate that our generator G is using context window slid over the samples to know the sequence patterns as shown in Figure 7, which can apprehend the metamorphic and dynamic code successfully. This method is already proven to be successful on previous studies.[73,74]

Apart from that, we split up our dataset into training and testing sets with a ratio of 60% to 40%, respectively. Remember that the test data was never used in the training process and is analogous to the zero-day malware samples. Test data is used only to evaluate the performance of the proposed architecture.

However, we did come across the false positive alarms. The interpretation and justification of these alarms need the visualization of malware and expert opinion for the analysis. This project is under execution in our laboratory and is thus currently not covered in this particular paper.

Our best combination of results is put in contrast to the most updated research techniques. It is evident from our results how well our model has performed in comparison to the referenced techniques. This comparison is summed up in Table 5. Apart from that the loss for generator and discriminator is shown in Figure 11 and confusion matrix is shown in Figure 12. Our model's accuracy is computed and diagnostics of our model are determined by way of ROC, precision versus recall curves, and F1 score. These are all demonstrated in Figure 13.

Notice that we make use of state-of-the-art models, ie, Peng et al,[84] Drebin,[49] RevealDroid,[85] and Nauman et al,[86] for the sake of comparison to our model. Interestingly, the work of Nair and Hinton[27] has managed to cross all the preceding results and has few results that are close to ours. The cost of calculating and getting rid of uncertainty from the large dataset such as the one we augmented is limiting their applicability in the scenarios involving businesses and naive users. Similarly, we observe that contributors of Nauman et al[86] accomplished the soaring high F1 score measure by sloping down the coverage to 0.976.

As opposed to this, we augmented a very large input space, which is the combination of three state-of-the-art datasets and then trained and tested our model on this huge volume of 20 GB. On ground, we were able to supersede the results of Nauman et al[86] as we fittingly acquainted our neural networks with early stopping in the training process, which guided

TABLE 5 Starting from the left, the first column provides the list of metrics, and the second column sums up the latest mechanism for Android Malware. The Evaluations in the latter columns (to the right) are the efforts of this research contribution

Metric	84	49	85	86	LSTM-GAN
Accuracy	-	-	0.858	0.99	0.991
Precision	-	-	0.892	0.992	0.99
F1 Score	-	-	0.874	0.996	0.99
Coverage	1.0	1.0	1.0	0.976	-
Support	-	-	-	-	provided
FPR	-	0.01	-	0.002	0.002
Detection rate	-	0.94	0.969	0.991	0.989
AUC	0.953	-	-	0.993	0.987

Abbreviations: AUC, area calculated under the receiver operating characteristic curves; FPR, false positive rate; LSTM-GAN, long short-term memory generative adversarial network.

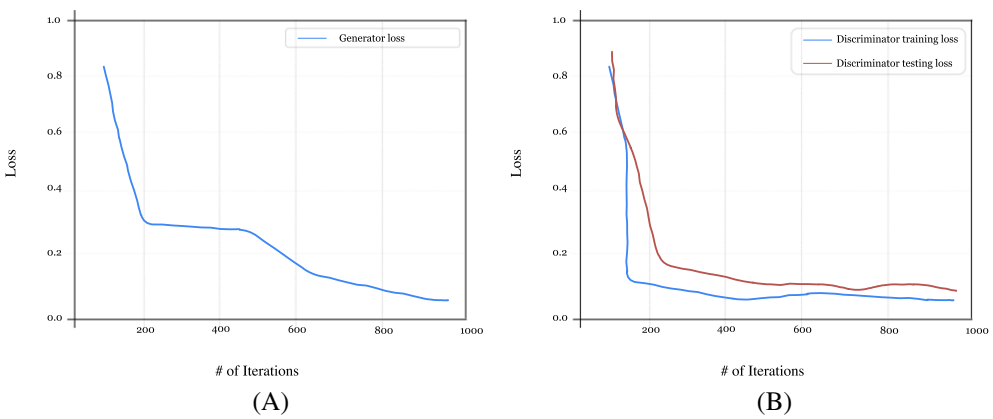

FIGURE 11 Loss for generative adversarial network. A, Generator loss over iterations; B, Discriminator loss over iterations

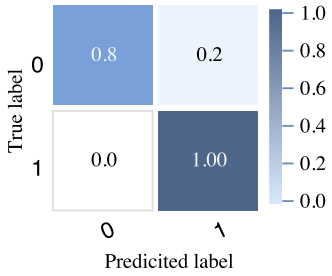

FIGURE 12 Confusion matrix with normalization of input

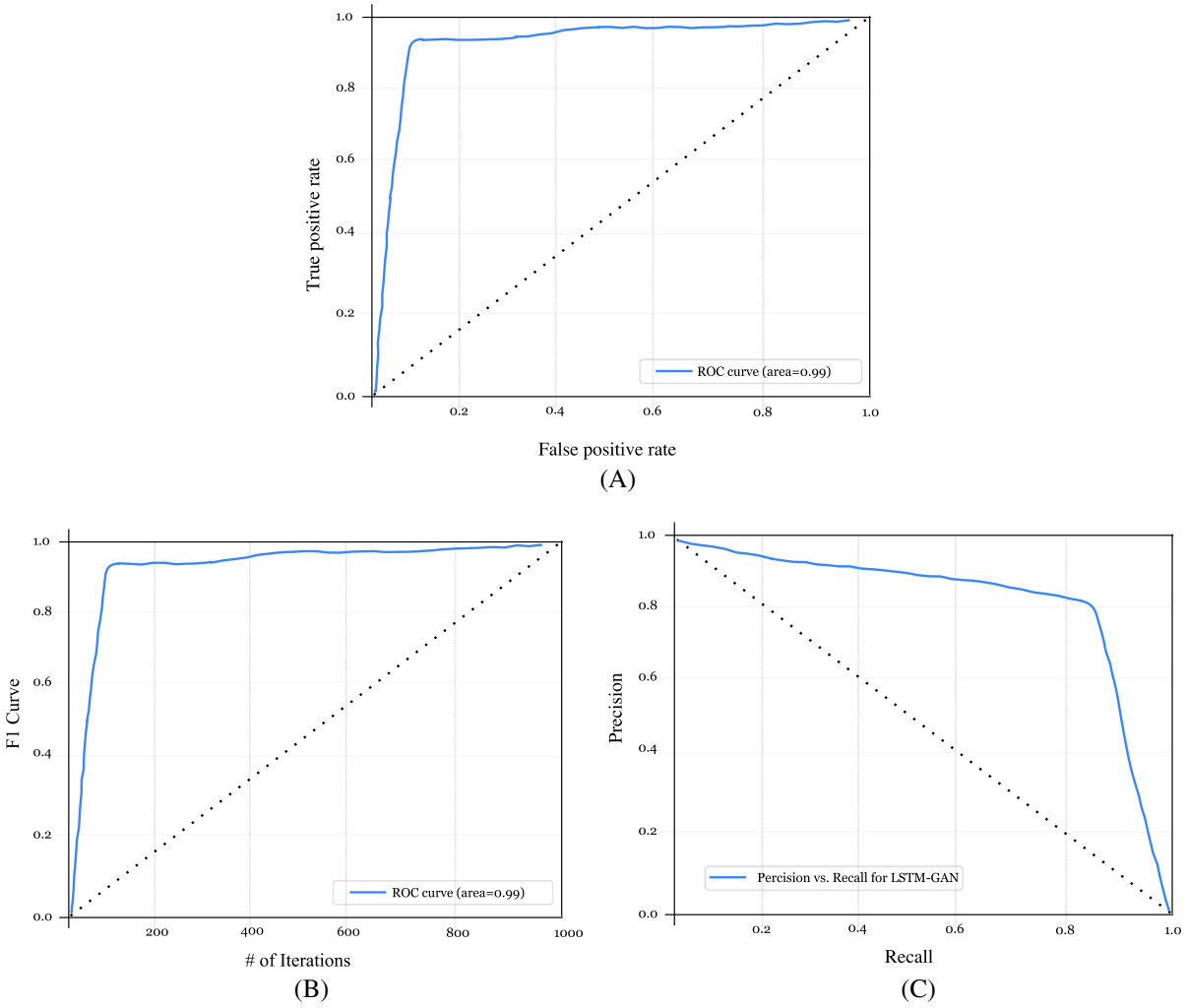

FIGURE 13 Receiver operating characteristic (ROC), F1 score, and precision versus recall curves for long short-term memory generative adversarial network (LSTM-GAN). A, Receiver operating characteristics (ROC) curve for LSTM-GAN; B, F1 score for LSTM-GAN; C, Precision versus recall for LSTM-GAN

us to deflect network convergence and avoid *overfitting*. In comparison, in Table 5, the models that did not report criteria or never exercised it are marked with a '-' symbol. Note that we have referred our work as LSTM-GAN in the said table.

For the reader's understanding and comprehension, we have provided the training and testing loss for the discriminator and training loss for generator in Figure 5. Additionally, we provide a snapshot of epochwise training facet in Figure 14 to have a better picture of what and how it happened during the training process. We have not provided the testing loss for Generator due to the lack of correlation and intuitiveness. In addition, we purposely did not report the coverage metric of our results since it causes an overhead and limits applicability.

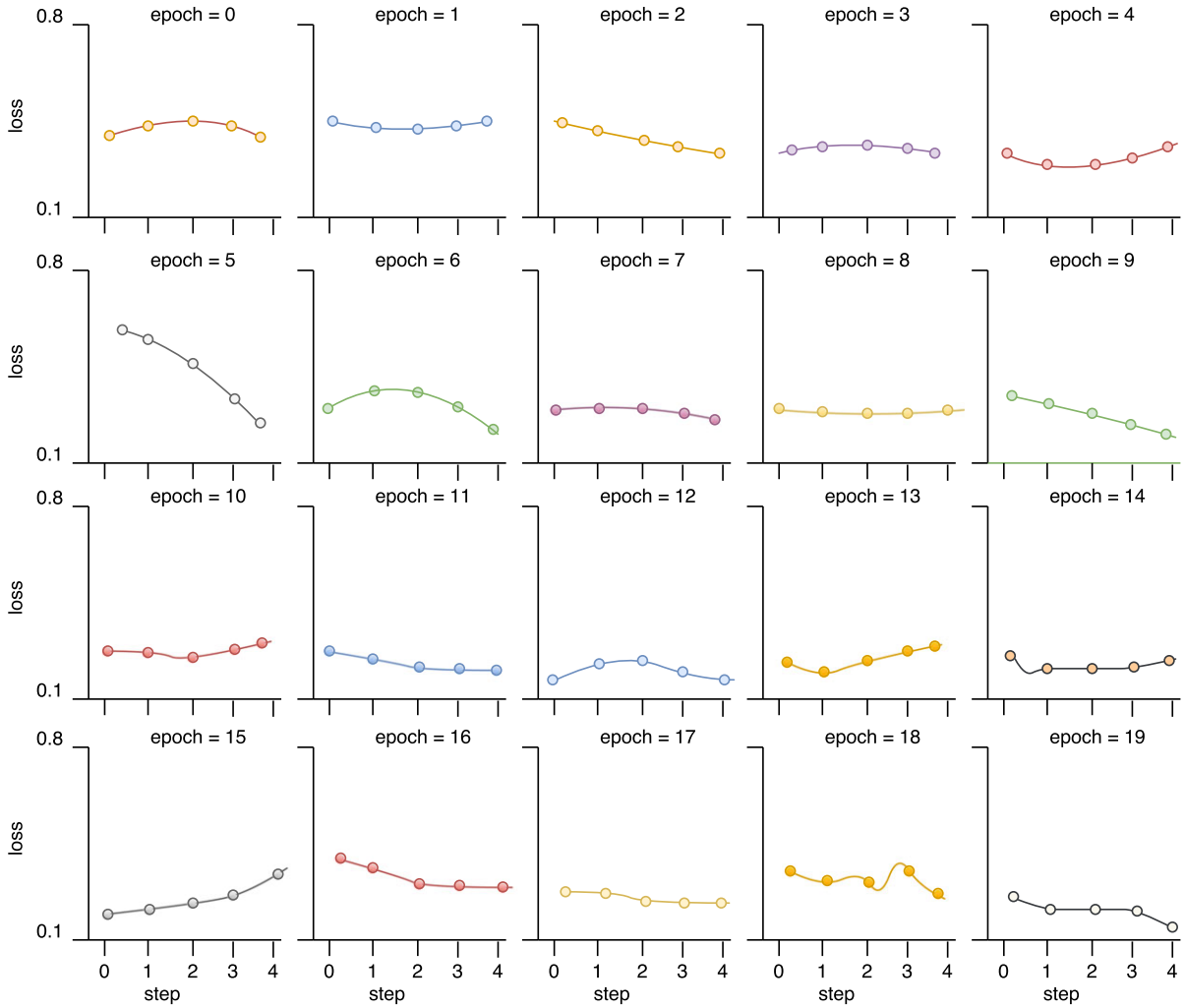

FIGURE 14 A grid facet for the visualization of epochwise training

GANs are a good fit for malware detection. To the best of our knowledge, this model is the first system that performs such a large-scale detection and analysis for application behavior in addition to zero-day malware detection with GANs. Our approach using only the unsupervised static features is a complementary fix to the mechanism using runtime features. Features learned and used by our model are unique and were able to apprehend the malicious apps in larger numbers as compared with the present one in the original dataset that was previously not labeled as malicious by some detection methods based on conventional mechanism.[49] GANs are implicit generative models, which are highly a perfect fit for large-scale data without assuming any prior distribution function. Due to this reason, the conventional machine learning mechanism is less friendly to very large-scale data such as ours.

Also, in comparison to other deep learning and generative models, GANs are optimized quickly and reach a saddle point successfully. This ability make GANs unique and suitable option for training and generating unique samples for evaluation.[22]

10.2 | Lexical dispersion plot

Lexical dispersion plots[81] allow us to determine the locations of a word in the text, ie, how many times the word (or multiple words) occurs from the beginning to the end of the text. Lexical dispersion plots can add an interesting dimension to malware analysis depending on how you choose to use it. In our case, the words are opcode binary occurrences in all of the Android applications (apks), as shown in Figure 15.

Lexical dispersion plots provide an alternative insight into the popularity of these technical words. Although a simple word count shows a relatively accurate picture of a word's prevalence in the particular application, it is also interesting to investigate the patterns in which these words appear.

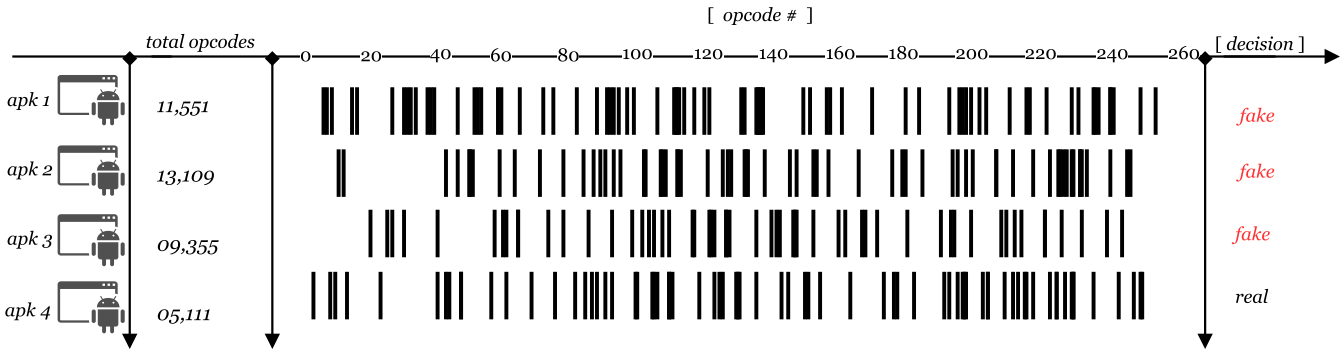

FIGURE 15 Lexical dispersion plot

To the left of Figure 15 is the apps given on *y*-axis of the dispersion plot and those on the *x*-axis appears the list of complete opcodes.

Highlighted instances of the plot show occurrences of the opcodes (bytecode sequences) in an application at a particular location. Collectively, these highlighted instances provide a useful insight and ground on which a particular apk is figured as real or fake.

11 | CONCLUSION AND FUTURE WORK

In this research work, we have studied Android malware and used a unique method for its detection via GANs, which is currently among the state-of-the-art deep learning architectures.

We have also exhibited the structure and layout of large input dataset comprising opcodes present in the bytecode of Android apks, which were pruned and fed to the *GANs* after converting them to a tensor shape. We not only shared our experimental results but also provide a comparison with existing state-of-the-art techniques. Arguably, our results match and supersede any of the existing methods.

We believe that this work is just a start of applying deep neural networks to the field of malware detection for smartphones like Android. In our work, we just touched and excavated the Android malware using static analysis of code. Nonetheless, this work could greatly be extended via dynamic malware analysis at bytecode level but also to application-level inspection. Certain malware may just be perceived by examining different aspects of an apk's payload. Our rationality is to judge how far we can go with our technique while maintaining simplicity in the input. Our work can definitely be extended in future while taking into the account of other elements in the payload of an app.

Furthermore, the inception of Bayesian neural networks[87] adds more wit to the neural networks and can vigorously be applied to the domain of malware analysis and code obfuscation. Our work provides reasonable evidence that deep learning has much more to offer as a toolset that is adequately satisfactory to cope with the escalating threat levels postured by malware creators to the smartphones ecology. With this work, we have a firm belief that it will lead the scientific community to put focus on pacifying threats faced by smartphones and hence help these platforms become more adaptable, accurate, and secure.

ACKNOWLEDGMENTS

This study was supported by Zayed University-UAE under RIF:R18051. We would like to express our deep feelings, for making us available with a handy GPU Tesla K40c, to the NVIDIA Corporation. This *CUDA* ready device was our only chance to complete the 24/7 training and testing phases. Furthermore, this exploration of malware detection would have not been conceivable without the datasets accessible free from any cost for education and research purposes. We appreciate and are thankful to Drebin,[49] AMD[64] and the VirshShare site[65] for maintaining these facilities up to date.

ORCID

Muhammad Amin https://orcid.org/0000-0003-3484-9793

REFERENCES

1. Zhou W, Zhou Y, Jiang X, Ning P. Detecting repackaged smartphone applications in third-party android marketplaces. In: Proceedings of the 2nd ACM Conference on Data and Application Security and Privacy (CODASPY); 2012; San Antonio, TX.
2. Unuchek R. Mobile malware evolution 2017. Kaspersky Lab. 2018. https://securelist.com/mobile-malware-review-2017/84139/. Accessed March 8, 2018.
3. The AV-Test Security Report. https://www.av-test.org/fileadmin/pdf/security_report/AV-TEST_Security_Report_2015-2016.pdf. Accessed April 3, 2018.
4. Mansfield-Devine S. Android architecture: attacking the weak points. *Network Security*. 2012;2012(10):5-12.
5. Nath HV, Mehtre BM. Static malware analysis using machine learning methods. In: *Recent Trends in Computer Networks and Distributed Systems Security: Second International Conference, SNDS 2014, Trivandrum, India, March 13-14, 2014, Proceedings*. Berlin, Germany: Springer-Verlag Berlin Heidelberg; 2014:440-450.
6. Fereidooni H, Conti M, Yao D, Sperduti A. ANASTASIA: ANdroid mAlware detection using STatic analySIs of applications. Paper presented at: 2016 8th IFIP International Conference on New Technologies, Mobility and Security (NTMS); 2016; Larnaca, Cyprus.
7. Zhou Y, Wang Z, Zhou W, Jiang X. Hey, you, get off of my market: detecting malicious apps in official and alternative android markets. Paper presented at: 19th Annual Network and Distributed System Security Symposium (NDSS); 2012; San Diego, CA.
8. Medvet E, Mercaldo F. Exploring the usage of topic modeling for android malware static analysis. Paper presented at: 2016 11th International Conference on Availability, Reliability and Security (ARES); 2016; Salzburg, Austria.
9. Grace M, Zhou Y, Zhang Q, Zou S, Jiang X. RiskRanker: scalable and accurate zero-day android malware detection. In: Proceedings of the 10th International Conference on Mobile Systems, Applications, and Services (MobiSys); 2012; Lake District, UK. http://doi.acm.org/10.1145/2307636.2307663
10. Derr E. The impact of third-party code on android app security. USENIX Association; 2018.
11. Ruiz F. Obfuscated malware discovered on Google play. McAfee. https://securingtomorrow.mcafee.com/mcafee-labs/obfuscated-malware-discovered-google-play/. Accessed February 3, 2018.
12. Suykens JA, Vandewalle J. Least squares support vector machine classifiers. *Neural Process Lett*. 1999;9(3):293-300.
13. Wang T, Wu JD, Coates A, Ng YA. End-to-end text recognition with convolutional neural networks. In: Proceedings of the 2012 21st International Conference on Pattern Recognition (ICPR); 2012; Tsukuba, Japan.
14. Cortes C, Vapnik V. Support-vector networks. *Machine Learning*. 1995;20(3):273-297.
15. Krizhevsky A, Sutskever I, Hinton EG. Imagenet classification with deep convolutional neural networks. In: *Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012*. Red Hook, NY: Curran Associates, Inc; 2012:1097-1105.
16. LeCun Y, Bengio Y. Convolutional networks for images, speech, and time series. In: *The Handbook of Brain Theory and Neural Networks*. Cambridge, MA: The MIT Press; 1995:276.
17. Deng L, Li J, Huang JT, et al. Recent advances in deep learning for speech research at Microsoft. Paper presented at: 2013 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP); 2013; Vancouver, Canada.
18. Farabet C, Couprie C, Najman L, LeCun Y. Learning hierarchical features for scene labeling. *IEEE Trans Pattern Anal Mach Intell*. 2013;35(8):1915-1929.
19. LeCun Y, Bengio Y, Hinton G. Deep learning. *Nature*. 2015;521(7553):436-444.
20. Goodfellow I, Pouget-Abadie J, Mirza M, et al. Generative adversarial nets. In: *Advances in Neural Information Processing Systems 27*. Red Hook, NY: Curran Associates, Inc; 2014:2672-2680.
21. Arora S, Ge R, Liang Y, Ma T, Zhang Y. Generalization and equilibrium in generative adversarial nets (GANs). arXiv preprint arXiv:1703.00573; 2017.
22. Salimans T, Goodfellow I, Zaremba W, et al. Improved techniques for training GANs. In: Lee DD, Sugiyama M, Luxburg UV, Guyon I, Garnett R, eds. *Advances in Neural Information Processing Systems 29*. Red Hook, NY: Curran Associates, Inc; 2016:2234-2242.
23. Cybenko G. Approximation by superpositions of a sigmoidal function. *Math Control Signals Syst*. 1989;2:303-314.
24. Biswas A, Shapiro V. Approximate distance fields with non-vanishing gradients. *Graphical Models*. 2004;66(3):133-159.
25. Bottou L. Large-scale machine learning with stochastic gradient descent. In: *Proceedings of COMPSTAT'2010: 19th International Conference on Computational Statistics Paris France, August 22-27, 2010 Keynote, Invited and Contributed Papers*. Berlin, Germany: Springer-Verlag Berlin Heidelberg; 2010:177-186.
26. Wazwaz AM. The tanh method: exact solutions of the sine-Gordon and the sinh-Gordon equations. *Appl Math Comput*. 2005;167:1196-1210.
27. Nair V, Hinton GE. Rectified linear units improve restricted Boltzmann machines. In: Proceedings of the 27th International Conference on Machine Learning (ICML); 2010; Haifa, Israel.
28. Maas AL, Hannun AY, Ng AY. Rectifier nonlinearities improve neural network acoustic models. In: Proceedings of the 30th International Conference on Machine Learning; 2013; Atlanta, GA.
29. Bridle J. Training stochastic model recognition algorithms as networks can lead to maximum mutual information estimation of parameters. In: Touretzky DS, ed. *Advances in Neural Information Processing Systems 2*. San Francisco, CA: Morgan Kaufmann Publishers Inc; 1990:211-217.
30. Bergstra J, Bengio Y. Random search for hyper-parameter optimization. *J Mach Learn Res*. 2012;13:281-305.

31. Zhang T. Solving large scale linear prediction problems using stochastic gradient descent algorithms. In: Proceedings of the 21st International Conference on Machine Learning (ICML); 2004; Banff, Canada.
32. Yang Z, Hu Z, Deng Y, Dyer C, Smola A. Neural machine translation with recurrent attention modeling. arXiv preprint arXiv:1607.05108; 2016.
33. Chung J, Gulcehre C, Cho K, Bengio Y. Gated feedback recurrent neural networks. arXiv preprint arXiv:1502.02367; 2015.
34. Graves A, Jaitly N, Mohamed AR. Hybrid speech recognition with deep bidirectional LSTM. Paper presented at: 2013 IEEE Workshop on Automatic Speech Recognition and Understanding (ASRU); 2013; Olomouc, Czech Republic.
35. Hochreiter S, Schmidhuber J. Long short-term memory. *Neural computation*. 1997;9(8):1735-1780.
36. Radford A, Metz L, Chintala S. Unsupervised representation learning with deep convolutional generative adversarial networks. arXiv preprint arXiv:1511.06434; 2015.
37. Zhang H, Xu T, Li H, et al. StackGAN: text to photo-realistic image synthesis with stacked generative adversarial networks. Paper presented at: 2017 IEEE International Conference on Computer Vision (ICCV); 2017; Venice, Italy.
38. Isola P, Zhu JY, Zhou T, Efros AA. Image-to-image translation with conditional adversarial networks. Paper presented at: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR); 2017; Honolulu, HI.
39. Karras T, Aila T, Laine S, Lehtinen J. Progressive growing of GANs for improved quality, stability, and variation. CoRR; 2017.
40. Zhu JY, Park T, Isola P, Efros AA. Unpaired image-to-image translation using cycle-consistent adversarial networks. Paper presented at: 2017 IEEE International Conference on Computer Vision (ICCV); 2017; Venice, Italy.
41. Reed S, Akata Z, Yan X, Logeswaran L, Schiele B, Lee H. Generative adversarial text to image synthesis. In: Proceedings of the 33rd International Conference on International Conference on Machine Learning - Volume 48 (ICML); 2016; New York, NY. http://dl.acm.org/citation.cfm?id=3045390.3045503
42. Jonathan H, Stefano E. Generative adversarial imitation learning. CoRR; 2016. http://adsabs.harvard.edu/abs/2016arXiv160603476H
43. de Oliveira L, Paganini M, Nachman B. Learning particle physics by example: location-aware generative adversarial networks for physics synthesis. *Comput Softw Big Sci*. 2017;1(1):4. https://doi.org/10.1007/s41781-017-0004-6
44. Odena A. Semi-supervised learning with generative adversarial networks. ArXiv e-prints; 2016. Provided by the SAO/NASA Astrophysics Data System.
45. Canfora G, Medvet E, Mercaldo F, Visaggio CA. Acquiring and analyzing app metrics for effective mobile malware detection. In: Proceedings of the 2016 ACM on International Workshop on Security And Privacy Analytics (IWSPA); 2016; New Orleans, LA. http://doi.acm.org/10.1145/2875475.2875481
46. Ali-Gombe A, Ahmed I, Richard III GG, Roussev V. AspectDroid: android app analysis system. In: Proceedings of the 6th ACM Conference on Data and Application Security and Privacy (CODASPY); 2016; New Orleans, LA.
47. Yuan Z, Lu Y, Xue Y. Droiddetector: android malware characterization and detection using deep learning. *Tsinghua Sci Technol*. 2016;21:114-123.
48. Karbab B, Debbabi M, Derhab A, Mouheb D. Android malware detection using deep learning on API method sequences. CoRR; 2017.
49. Arp D, Spreitzenbarth M, Hübner M, Gascon H, Rieck K, Siemens C. Drebin: effective and explainable detection of android malware in your pocket. In: Proceedings of the Annual Symposium on Network and Distributed System Security (NDSS); 2014; San Diego, CA.
50. Karbab EB, Debbabi M, Derhab A, Mouheb D. Cypider: building community-based cyber-defense infrastructure for android malware detection. In: Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC); 2016; Los Angeles, CA. http://doi.acm.org/10.1145/2991079.2991124
51. Mariconti E, Onwuzurike L, Andriotis P, De Cristofaro E, Ross G, Stringhini G. MaMaDroid: detecting android malware by building Markov chains of behavioral models. Paper presented at: 24th Annual Network and Distributed System Security Symposium (NDSS). 2017; San Diego, CA.
52. Kim J, Kim TG, Im EG. Structural information based malicious app similarity calculation and clustering. In: Proceedings of the 2015 Conference on Research in Adaptive and Convergent Systems; 2015; Prague, Czech Republic. http://doi.acm.org/10.1145/2811411.2811545
53. Ali-Gombe A, Ahmed I, Richard III GG, Roussev V. OpSeq: android malware fingerprinting. In: Proceedings of the 5th Program Protection and Reverse Engineering Workshop (PPREW-5); 2015; Los Angeles, CA. http://doi.acm.org/10.1145/2843859.2843860
54. Hou S, Saas A, Chen L, Ye Y. Deep4MalDroid: a deep learning framework for android malware detection based on Linux kernel system call graphs. Paper presented at: 2016 IEEE/WIC/ACM International Conference on Web Intelligence Workshops (WIW); 2016; Omaha, NE.
55. Goodfellow JI. NIPS 2016 tutorial: generative adversarial networks. CoRR; 2017. https://arxiv.org/abs/1701.00160
56. Deya A, Panloup F, Tindel S. Rate of convergence to equilibrium of fractional driven stochastic differential equations with rough multiplicative noise. *Ann Probab*. 2019;47:464-518.
57. Bengio Y, Laufer É, Alain G, Yosinski J. Deep generative stochastic networks trainable by backprop. In: Proceedings of the 31st International Conference on Machine Learning; 2014; Beijing, China.
58. Salimans T, Karpathy A, Chen X, Kingma DP. PixelCNN++: improving the PixelCNN with discretized logistic mixture likelihood and other modifications. CoRR; 2017. http://adsabs.harvard.edu/abs/2017arXiv170105517S
59. Kingma PD, Salimans T, Józefowicz R, Chen X, Sutskever I, Welling M. Improving variational autoencoders with inverse autoregressive flow. In: *Advances in Neural Information Processing Systems 2016*. Red Hook, NY: Curran Associates; 2016:4743-4751.
60. Rapoport A. *Two-Person Game Theory*. Ann Arbor, MI: University of Michigan Press; 1973.

61. Fuglede B, Topsoe F. Jensen-Shannon divergence and Hilbert space embedding. Paper presented at: International Symposium on Information Theory (ISIT); 2004; Chicago, IL.
62. Erhan D, Bengio Y, Courville A, Manzagol PA, Vincent P, Bengio S. Why does unsupervised pre-training help deep learning? *J Mach Learn Res*. 2010;11:625-660.
63. Abad EC, García MJG, Blázquez RR, Bustos-Caballero A, García-Prada JC. Using an Android app for teaching mechanism and machine theory. In: *New Trends in Educational Activity in the Field of Mechanism and Machine Theory*. Cham, Switzerland: Springer Nature Switzerland AG; 2019:103-110.
64. Li Y, Jang J, Hu X, Ou X. Android malware clustering through malicious payload mining. In: *Research in Attacks, Intrusions, and Defenses: 20th International Symposium, RAID 2017, Atlanta, GA, USA, September 18-20, 2017, Proceedings*. Cham, Switzerland: Springer International Publishing AG; 2017:192-214.
65. VXShare: VirusShare. https://virusshare.com. Accessed January 3, 2017.
66. VXShare: VirusShare labeling. https://github.com/seymour1/label-virusshare. Accessed October 25, 2018.
67. Forrest S, Hofmeyr SA, Somayaji A, Longstaff TA. A sense of self for Unix processes. In: Proceedings of the 1996 IEEE Symposium on Security and Privacy; 1996; Oakland, CA.
68. Avizienis A, Laprie JC, Randell B, Landwehr C. Basic concepts and taxonomy of dependable and secure computing. *IEEE Trans Dependable Secure Comput*. 2004;1:11-33.
69. Shu X, Araujo F, Schales DL, et al. Threat intelligence computing. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS); 2018; Toronto, Canada.
70. Chawla A, Lee B, Fallon S, Jacob P. Host based intrusion detection system with combined CNN/RNN model. In: *ECML PKDD 2018 Workshops: Nemesis 2018, UrbReas 2018, SoGood 2018, IWAISe 2018, and Green Data Mining 2018, Dublin, Ireland, September 10-14, 2018, Proceedings*. Cham, Switzerland: Springer Nature Switzerland AG; 2018:149-158.
71. Mikolov T, Chen K, Corrado G, Dean J. Efficient estimation of word representations in vector space. CoRR; 2013.
72. Pennington J, Socher R, Manning CD. GloVe: global vectors for word representation. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP); 2014; Doha, Qatar.
73. Alam S, Sogukpinar I, Traore I, Horspool RN. Sliding window and control flow weight for metamorphic malware detection. *J Comput Virol Hacking Tech*. 2015;11:75-88.
74. Mueller J, Thyagarajan A. Siamese recurrent architectures for learning sentence similarity. In: Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI); 2016; Phoenix, AZ. https://dl.acm.org/citation.cfm?id=3016291
75. Jones E, Oliphant T, Peterson P. SciPy: open source scientific tools for Python. 2001. www.scipy.org
76. McKinney W. Data structures for statistical computing in python. In: Proceedings of the 9th Python in Science Conference; 2010; Austin, TX.
77. Bedini A. HDF5 for Python. http://www.h5py.org
78. Hunter JD. Matplotlib: a 2D graphics environment. *Comput Sci Eng*. 2007;9:90-95.
79. Waskom M, Botvinnik O, Hobson P, et al. seaborn: v0.7.0 (January 2016). 2016. https://doi.org/10.5281/zenodo.45133
80. Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: machine learning in python. *J Mach Learn Res*. 2011;12:2825-2830.
81. Loper E, Bird S. NLTK: the natural language toolkit. In: Proceedings of the ACL-02 Workshop on Effective Tools and Methodologies for Teaching Natural Language Processing and Computational Linguistics - Volume 1 (ETMTNLP); 2002; Philadelphia, PA.
82. Sak H, Senior A, Beaufays F. Long short-term memory recurrent neural network architectures for large scale acoustic modeling. Paper presented at: 15th Annual Conference of the International Speech Communication Association; 2014; Singapore.
83. Kingma DP, Ba J. Adam: a method for stochastic optimization. arXiv preprint arXiv:1412.6980; 2014.
84. Peng H, Gates C, Sarma B, et al. Using probabilistic generative models for ranking risks of android apps. In: Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS); 2012; Raleigh, NC.
85. Garcia J, Hammad M, Pedrood B, Bagheri-Khaligh A, Malek S. *Obfuscation-Resilient, Efficient, and Accurate Detection and Family Identification of Android Malware*. Technical reports. Fairfax, VA: George Mason University; 2015.
86. Nauman M, Tanveer TA, Khan S, Syed TA. Deep neural architectures for large scale android malware analysis. *Cluster Computing*. 2018;21:569-588.
87. Hernández-Lobato JM, Adams RP. Probabilistic backpropagation for scalable learning of Bayesian neural networks. arXiv preprint arXiv:1502.05336; 2015.

How to cite this article: Amin M, Shah B, Sharif A, Ali T, Kim K-lL, Anwar S. Android malware detection through generative adversarial networks. *Trans Emerging Tel Tech*. 2019;e3675. https://doi.org/10.1002/ett.3675